The Haira Programming Language

Language Specification

Version 1.0 Draft

Haira Team

February 4, 2026

Document Version: 1.0 Draft
Language Version: 1.0
Last Updated: February 4, 2026

Contents

Preface

Haira is a modern, compiled programming language designed for the AI era. It combines the simplicity of Go, the safety of Rust, and the expressiveness of modern functional languages, while introducing first-class AI integration for code generation.

Design Philosophy

1. **Explicit is better than implicit** – All dependencies, AI invocations, and behaviors are clearly stated in code.
2. **Local-first AI** – AI code generation happens at compile time using local models (llama.cpp, Ollama) with optional cloud fallback.
3. **Go-style simplicity** – Familiar error handling, explicit imports, and straightforward syntax.
4. **Zero-cost abstractions** – High-level constructs compile to efficient native code.
5. **Compile-time guarantees** – No runtime AI dependencies; binaries are standalone.

Target Domains

Haira is designed for:

- Systems programming
- Command-line tools
- AI-assisted application development
- Web services and APIs
- Data processing pipelines

Implementation Roadmap

This specification is organized so that each chapter can be implemented incrementally:

Release	Chapters	Features
v0.1	1–3	Lexer, parser, basic types
v0.2	4–5	Expressions, statements
v0.3	6–7	Functions, error handling
v0.4	8	Concurrency primitives
v0.5	9–10	Modules, standard library

Release	Chapters	Features
v0.6	11–12	Advanced features, full grammar
v1.0	13–15	AI integration, compiler completion

Part I

Core Language

Chapter 1

Overview

Haira is a statically-typed, compiled programming language that brings AI-assisted code generation into the compilation pipeline. It compiles to native binaries via Cranelift, with no runtime dependencies.

1.1 Hello, World

```
import "io"

fn main() {
    io.println("Hello, World!")
}
```

To compile and run:

```
$ haira build hello.haira
$ ./hello
Hello, World!
```

1.2 Language Characteristics

1.2.1 Explicit Imports

All dependencies are explicitly declared at the top of each file:

```
import "io"
import "json"
import db from "haira/postgres"
```

1.2.2 Go-Style Error Handling

Functions that can fail return a tuple of result and error:

```
import "io"
import "fs"

fn main() {
    content, err = fs.read_file("config.json")
```

```
    if err != nil {
        io.println("Error: " + err.message)
        return
    }
    io.println(content)
}
```

1.2.3 Explicit AI Blocks

AI-generated code is explicit, using the **ai** keyword:

```
import "io"

struct User {
    name: string
    email: string
    posts: []Post
}

ai summarize_user(user: User) -> string {
    Generate a brief, human-readable summary of this user
    including their name and post count.
}

fn main() {
    user = User{
        name: "Alice",
        email: "alice@example.com",
        posts: []
    }
    io.println(summarize_user(user))
}
```

Note

AI blocks are processed at **compile time**. The generated code is cached and included in the binary. There are no AI dependencies at runtime.

1.2.4 Local-First AI

AI code generation uses local models by default:

1. **llama.cpp** – Primary backend, runs local GGUF models
2. **Ollama** – Secondary, connects to local Ollama server
3. **Cloud APIs** – Optional fallback (Anthropic, OpenAI)

Configuration in `haira.toml`:

```
[ai]
backend = "llama.cpp"
model_path = "~/.haira/models/codellama-7b.gguf"
```

1.3 Type System

HAIRA uses structural typing with type inference for local variables:

```
// Type inferred as int
count = 42

// Explicit type annotation
name: string = "Haira"

// Struct definition
struct Point {
    x: float
    y: float
}

// Option type (nullable)
maybe_value: int? = nil

// Arrays and maps
numbers: []int = [1, 2, 3]
scores: [string:int] = ["alice": 100, "bob": 95]
```

1.4 Control Flow

```
import "io"

fn main() {
    // If-else
    x = 10
    if x > 5 {
        io.println("Large")
    } else {
        io.println("Small")
    }

    // For loop with range
    for i in 0..5 {
        io.println(i)
    }

    // While loop
    n = 0
    while n < 3 {
        io.println(n)
        n = n + 1
    }

    // Match expression
    status = 200
    match status {
        200 => io.println("OK")
        404 => io.println("Not Found")
        500 => io.println("Server Error")
        _ => io.println("Unknown")
```

```
    }
}
```

1.5 Functions

Functions are declared with **fn**:

```
fn add(a: int, b: int) -> int {
    return a + b
}

// Multiple return values
fn divide(a: int, b: int) -> (int, Error?) {
    if b == 0 {
        return 0, Error{message: "division by zero"}
    }
    return a / b, nil
}

// Closures
fn make_adder(n: int) -> fn(int) -> int {
    return fn(x: int) -> int {
        return x + n
    }
}
```

1.6 Concurrency

HAIRA provides Go-style concurrency with **spawn** and channels:

```
import "io"

fn main() {
    ch = chan<int>(10)  // Buffered channel

    spawn {
        for i in 0..5 {
            ch <- i
        }
        close(ch)
    }

    for value in ch {
        io.println(value)
    }
}
```

1.7 Pipe Operator

The pipe operator | enables functional composition:

```
import "io"
```

```
import "string"
import "array"

fn main() {
    result = "  hello, world  "
        | string.trim
        | string.to_upper
        | string.split(", ")
        | array.first

    io.println(result)  // "HELLO"
}
```

1.8 A Complete Example

```
import "io"
import "http"
import "json"

struct User {
    id: int
    name: string
    email: string
}

ai validate_email(email: string) -> (bool, string?) {
    Validate the email format and return whether it's valid.
    If invalid, return an error message explaining why.
}

fn create_user(name: string, email: string) -> (User, Error?) {
    valid, reason = validate_email(email)
    if not valid {
        return User{}, Error{message: reason or "Invalid email"}
    }

    user = User{
        id: generate_id(),
        name: name,
        email: email
    }

    return user, nil
}

fn main() {
    user, err = create_user("Alice", "alice@example.com")
    if err != nil {
        io.println("Failed: " + err.message)
        return
    }

    io.println("Created user: " + user.name)
}

```

```
fn generate_id() -> int {
    // Simple ID generation
    return 1
}
```

1.9 Implementation Status

This specification documents the complete Haira language. Implementation proceeds incrementally:

Feature	Chapter	Target Release
Lexer & Parser	2–3	v0.1
Expressions & Statements	4–5	v0.2
Functions	6	v0.3
Error Handling	7	v0.3
Concurrency	8	v0.4
Modules & Stdlib	9–10	v0.5
Advanced Features	11	v0.6
AI Integration	12–13	v1.0

Table 1.1: Implementation roadmap by chapter

Implementation Note

Features not yet implemented will produce clear error messages indicating the target release version.

Chapter 2

Lexical Structure

This chapter defines the lexical grammar of Haira: the rules for converting source text into tokens.

2.1 Source Encoding

Haira source files are encoded in UTF-8. The file extension is `.haira`.

```
hello.haira
utils/string_helpers.haira
```

2.2 Line Structure

Haira is a line-oriented language. Statements are terminated by newlines, not semicolons.

```
x = 1
y = 2
z = x + y
```

2.2.1 Line Continuation

Lines can be continued with a trailing backslash or when the line ends with an operator or open bracket:

```
// Explicit continuation
long_result = very_long_function_name(arg1, arg2) \
    + another_function(arg3)

// Implicit continuation (ends with operator)
total = first_value +
    second_value +
    third_value

// Implicit continuation (open bracket)
users = [
    "alice",
    "bob",
    "charlie"
]
```

2.3 Comments

2.3.1 Single-Line Comments

```
// This is a single-line comment
x = 42  // Inline comment
```

2.3.2 Multi-Line Comments

```
/*
    This is a multi-line comment.
    It can span multiple lines.
*/
```

Multi-line comments can be nested:

```
/*
    Outer comment
    /* Nested comment */
    Still in outer comment
*/
```

2.3.3 Documentation Comments

```
/// This is a documentation comment for the following item.
/// It can span multiple lines.
fn calculate_area(width: float, height: float) -> float {
    return width * height
}
```

2.4 Tokens

The lexer produces the following token types:

1. Keywords
2. Identifiers
3. Literals (numbers, strings, booleans)
4. Operators
5. Delimiters

2.5 Keywords

The following identifiers are reserved as keywords:

`ai`	`and`	`as`	`break`	`catch`
`chan`	`continue`	`defer`	`else`	`enum`
`false`	`fn`	`for`	`from`	`if`
`import`	`in`	`match`	`nil`	`not`
`or`	`return`	`select`	`spawn`	`struct`
`true`	`try`	`type`	`while`	

Table 2.1: Reserved keywords

Note

The keywords **`ai`**, **`catch`**, **`defer`**, **`select`**, **`spawn`**, and **`try`** are reserved for future language features. Using them will produce a clear error message.

2.6 Identifiers

Identifiers name variables, functions, types, and modules.

```
identifier      = letter { letter | digit | "_" } ;
letter          = "a".."z" | "A".."Z" | "_" ;
digit           = "0".."9" ;
```

Valid identifiers:

```
x
name
userName
user_name
_private
Point3D
MAX_SIZE
```

Invalid identifiers:

```
3d_point     // Cannot start with digit
my-name      // Hyphens not allowed
class        // Reserved keyword (if added)
```

2.6.1 Naming Conventions

Entity	Convention
Variables	`snake_case`
Functions	`snake_case`
Types/Structs	`PascalCase`
Constants	`SCREAMING_SNAKE_CASE`
Modules	`snake_case`

Table 2.2: Naming conventions

2.7 Literals

2.7.1 Integer Literals

```
integer_literal = decimal_literal
                | hex_literal
                | octal_literal
                | binary_literal ;

decimal_literal = digit { digit | "_" } ;
hex_literal     = "0" ("x"|"X") hex_digit { hex_digit | "_" } ;
octal_literal   = "0" ("o"|"O") octal_digit { octal_digit | "_" } ;
binary_literal  = "0" ("b"|"B") binary_digit { binary_digit | "_" } ;

hex_digit       = digit | "a".."f" | "A".."F" ;
octal_digit     = "0".."7" ;
binary_digit    = "0" | "1" ;
```

Examples:

```
decimal = 42
with_underscores = 1_000_000
hex = 0xFF
octal = 0o755
binary = 0b1010_1100
```

Note

All integer literals are of type `int` (64-bit signed).

2.7.2 Floating-Point Literals

```
float_literal = decimal_literal "." decimal_literal [ exponent ]
              | decimal_literal exponent ;

exponent = ("e"|"E") ["+"|"-"] decimal_literal ;
```

Examples:

```
pi = 3.14159
scientific = 6.022e23
negative_exp = 1.6e-19
```

2.7.3 Boolean Literals

```
is_valid = true
is_empty = false
```

2.7.4 String Literals

```
string_literal = '"' { string_char } '"' ;
string_char    = any_char - ('"' | '\' | newline)
               | escape_sequence ;
```

Escape sequences:

Sequence	Meaning
\n	Newline (LF)
\r	Carriage return (CR)
\t	Horizontal tab
\\	Backslash
\"	Double quote
\0	Null character
\xNN	Hex byte (00-FF)
\u{NNNN}	Unicode code point

Table 2.3: String escape sequences

Examples:

```
simple = "Hello, World!"
with_escapes = "Line 1\nLine 2"
unicode = "Hello \u{1F44B}"  // Wave emoji
```

2.7.5 String Interpolation

Strings can contain interpolated expressions using `${...}`:

```
name = "Alice"
age = 30
greeting = "Hello, ${name}! You are ${age} years old."
```

Implementation Note

String interpolation is desugared to concatenation during parsing:

```
greeting = "Hello, " + name + "! You are " +
    to_string(age) + " years old."
```

2.7.6 Raw Strings

Raw strings preserve backslashes literally:

```
regex_pattern = r"^\d{3}-\d{4}$"
windows_path = r"C:\Users\Alice\Documents"
```

2.7.7 Multi-Line Strings

```
query = """
    SELECT *
    FROM users
    WHERE active = true
"""
```

The leading whitespace common to all lines is stripped.

2.7.8 Nil Literal

```
empty: User? = nil
```

2.8 Operators

2.8.1 Arithmetic Operators

Operator	Description
`+`	Addition
`-`	Subtraction (or unary negation)
`*`	Multiplication
`/`	Division
`%`	Modulo (remainder)

2.8.2 Comparison Operators

Operator	Description
`==`	Equal
`!=`	Not equal
`<`	Less than
`>`	Greater than
`<=`	Less than or equal
`>=`	Greater than or equal

2.8.3 Logical Operators

Operator	Description
`and`, `&&`	Logical AND
`or`, `||`	Logical OR
`not`, `!`	Logical NOT

Operator	Description
`=`	Assignment
`+=`	Add and assign
`-=`	Subtract and assign
`*=`	Multiply and assign
`/=`	Divide and assign

2.8.4 Assignment Operators

2.8.5 Other Operators

Operator	Description
`|`	Pipe (function composition)
`..`	Range (exclusive end)
`..=`	Range (inclusive end)
`<-`	Channel send
`->`	Function return type
`=>`	Match arm

2.9 Delimiters

Delimiter	Usage
`( )`	Grouping, function calls, tuples
`[ ]`	Array literals, indexing
`{ }`	Blocks, struct literals, maps
`,`	Separator
`:`	Type annotations, map entries
`.`	Member access
`?`	Optional type marker

2.10 Whitespace

Whitespace (spaces, tabs) is used to separate tokens but is otherwise insignificant, except for indentation in multi-line strings.

```
x=1+2          // Valid
x = 1 + 2      // Also valid, preferred for readability
```

2.11 Lexical Grammar Summary

```
token = keyword
      | identifier
      | integer_literal
      | float_literal
```

```
      | string_literal
      | operator
      | delimiter ;

keyword = "ai" | "and" | "as" | "break" | "catch" | "chan"
        | "continue" | "defer" | "else" | "enum" | "false"
        | "fn" | "for" | "from" | "if" | "import" | "in"
        | "match" | "nil" | "not" | "or" | "return" | "select"
        | "spawn" | "struct" | "true" | "try" | "type" |
           "while" ;

identifier = letter { letter | digit | "_" } ;

operator = "+" | "-" | "*" | "/" | "%"
         | "==" | "!=" | "<" | ">" | "<=" | ">="
         | "and" | "or" | "not" | "&&" | "||" | "!"
         | "=" | "+=" | "-=" | "*=" | "/="
         | "|" | ".." | "..=" | "<-" | "->" | "=>" ;

delimiter = "(" | ")" | "[" | "]" | "{" | "}"
          | "," | ":" | "." | "?" ;
```

Chapter 3

Types

Haira is a statically-typed language with structural typing and type inference for local variables. All types are known at compile time.

3.1 Type System Overview

- **Static typing** – All types resolved at compile time
- **Structural typing** – Type compatibility based on structure, not name
- **Type inference** – Local variables infer types from initializers
- **Explicit annotations** – Required for function signatures
- **No null** – Option types (`T?`) replace null pointers

3.2 Primitive Types

3.2.1 Integer Types

Type	Size	Range
`int`	64 bits	-2^{63} to $2^{63}-1$
`i8`	8 bits	-128 to 127
`i16`	16 bits	-32768 to 32767
`i32`	32 bits	-2^{31} to $2^{31}-1$
`i64`	64 bits	-2^{63} to $2^{63}-1$
`u8`	8 bits	0 to 255
`u16`	16 bits	0 to 65535
`u32`	32 bits	0 to $2^{32}-1$
`u64`	64 bits	0 to $2^{64}-1$

Table 3.1: Integer types

```
count: int = 42
byte: u8 = 255
offset: i32 = -100
```

Note

The default `int` is an alias for `i64`. Integer literals without explicit type default to `int`.

3.2.2 Floating-Point Types

Type	Size	Precision
`float`	64 bits	IEEE 754 double
`f32`	32 bits	IEEE 754 single
`f64`	64 bits	IEEE 754 double

Table 3.2: Floating-point types

```
pi: float = 3.14159
temperature: f32 = 98.6
```

3.2.3 Boolean Type

```
is_valid: bool = true
is_empty: bool = false
```

Boolean values are used in conditionals and logical expressions. Only `true` and `false` are valid boolean values—there is no implicit conversion from other types.

3.2.4 String Type

Strings are immutable sequences of UTF-8 encoded bytes:

```
name: string = "Haira"
greeting: string = "Hello, " + name
```

String operations:

```
import "string"

s = "Hello, World!"
length = string.len(s)          // 13
upper = string.to_upper(s)      // "HELLO, WORLD!"
parts = string.split(s, ", ") // ["Hello", "World!"]
```

3.3 Composite Types

3.3.1 Arrays

Arrays are ordered, homogeneous, dynamically-sized collections:

```
// Array literal
numbers: []int = [1, 2, 3, 4, 5]
```

```
3
4  // Empty array
5  empty: []string = []
6
7  // Array operations
8  import "array"
9
10 first = numbers[0]              // 1
11 length = array.len(numbers)    // 5
12 numbers = array.push(numbers, 6)
```

```
1  array_type = "[" "]" type ;
```

3.3.2 Maps

Maps are unordered key-value collections:

```
1  // Map literal
2  scores: [string:int] = [
3      "alice": 100,
4      "bob": 95,
5      "charlie": 87
6  ]
7
8  // Empty map
9  empty: [string:User] = [:]
10
11 // Map operations
12 alice_score = scores["alice"]     // 100
13 scores["david"] = 92              // Insert
```

```
1  map_type = "[" type ":" type "]" ;
```

Note

Map keys must be comparable types: primitives, strings, or structs containing only comparable fields.

3.3.3 Tuples

Tuples are fixed-size, heterogeneous collections:

```
1  // Tuple type
2  point: (int, int) = (10, 20)
3
4  // Accessing elements
5  x = point.0  // 10
6  y = point.1  // 20
7
8  // Multiple return values use tuples
9  fn divide(a: int, b: int) -> (int, int) {
10     return a / b, a % b
```

```
}

quotient, remainder = divide(17, 5)
```

```
tuple_type = "(" type { "," type } ")" ;
```

3.4 Option Types

Option types represent values that may or may not exist, replacing null pointers:

```
// Optional value
maybe_name: string? = nil
maybe_age: int? = 25

// Checking for nil
if maybe_name != nil {
    io.println(maybe_name)
}

// Default value with 'or'
name = maybe_name or "Unknown"
```

```
option_type = type "?" ;
```

3.4.1 Unwrapping Options

```
value: int? = get_value()

// Safe unwrap with default
result = value or 0

// Conditional unwrap
if value != nil {
    // value is automatically unwrapped here
    io.println(value)
}

// Match on option
match value {
    nil => io.println("No value")
    v => io.println("Got: " + to_string(v))
}
```

3.5 Struct Types

Structs are named product types with named fields:

```
struct User {
    id: int
```

```
    name: string
    email: string
    active: bool
}

// Creating a struct
user = User{
    id: 1,
    name: "Alice",
    email: "alice@example.com",
    active: true
}

// Accessing fields
io.println(user.name)

// Updating fields (structs are mutable by default)
user.active = false
```

3.5.1 Struct Methods

Methods can be defined on structs:

```
struct Rectangle {
    width: float
    height: float
}

fn (r: Rectangle) area() -> float {
    return r.width * r.height
}

fn (r: Rectangle) perimeter() -> float {
    return 2 * (r.width + r.height)
}

// Usage
rect = Rectangle{width: 10.0, height: 5.0}
io.println(rect.area())      // 50.0
io.println(rect.perimeter()) // 30.0
```

3.5.2 Nested Structs

```
struct Address {
    street: string
    city: string
    country: string
}

struct Person {
    name: string
    address: Address
}

```

```
person = Person{
    name: "Bob",
    address: Address{
        street: "123 Main St",
        city: "New York",
        country: "USA"
    }
}

io.println(person.address.city)  // "New York"
```

3.6 Enum Types

Enums define a type with a fixed set of named values:

```
enum Color {
    Red,
    Green,
    Blue
}

enum Status {
    Pending,
    Active,
    Completed,
    Failed
}

// Usage
color: Color = Color.Red
status: Status = Status.Active

match color {
    Color.Red => io.println("Stop")
    Color.Green => io.println("Go")
    Color.Blue => io.println("Info")
}
```

3.6.1 Enums with Associated Data

Enums can carry associated data (tagged unions):

```
enum Result<T> {
    Ok(T),
    Err(Error)
}

enum Message {
    Text(string),
    Number(int),
    Pair(int, int)
}

// Usage
msg: Message = Message.Text("Hello")
```

```
14
15 match msg {
16     Message.Text(s) => io.println("Text: " + s)
17     Message.Number(n) => io.println("Number: " + to_string(n))
18     Message.Pair(a, b) => io.println("Pair: " + to_string(a) +
           ", " + to_string(b))
19 }
```

3.7 Type Aliases

Type aliases create alternative names for existing types:

```
1 type UserId = int
2 type UserMap = [string:User]
3 type Handler = fn(Request) -> Response
4
5 // Usage
6 id: UserId = 42
7 users: UserMap = [:]
```

3.8 Function Types

Functions are first-class values with types:

```
1  // Function type syntax
2  type Comparator = fn(int, int) -> bool
3  type Transformer = fn(string) -> string
4  type Callback = fn()
5
6  // Higher-order functions
7  fn apply(f: fn(int) -> int, x: int) -> int {
8      return f(x)
9  }
10
11 fn double(n: int) -> int {
12     return n * 2
13 }
14
15 result = apply(double, 21)  // 42
```

3.9 Generic Types

Generic types parameterize over other types:

```
1 // Generic struct
2 struct Pair<T, U> {
3     first: T
4     second: U
5 }
6
7 // Usage
8 pair: Pair<int, string> = Pair{first: 1, second: "one"}
```

```
// Generic function
fn swap<T, U>(p: Pair<T, U>) -> Pair<U, T> {
    return Pair{first: p.second, second: p.first}
}

swapped = swap(pair)  // Pair<string, int>
```

3.9.1 Type Constraints

Generics can be constrained to types implementing certain behaviors:

```
// Constraint definition
constraint Comparable {
    fn compare(other: Self) -> int
}

// Constrained generic
fn max<T: Comparable>(a: T, b: T) -> T {
    if a.compare(b) > 0 {
        return a
    }
    return b
}

// Multiple constraints
fn process<T: Comparable + Printable>(value: T) {
    io.println(value.to_string())
}
```

3.10 The Error Type

The built-in `Error` type represents errors:

```
struct Error {
    message: string
    code: int?
    source: Error?
}

// Creating errors
err = Error{message: "file not found"}
err_with_code = Error{message: "permission denied", code: 403}

// Error chaining
wrapped = Error{
    message: "failed to read config",
    source: err
}
```

3.11 Type Inference

Haira infers types for local variables from their initializers:

```
// Types inferred
x = 42                // int
pi = 3.14             // float
name = "Haira"        // string
active = true         // bool
numbers = [1, 2, 3] // []int

// Explicit annotation when needed
count: i32 = 100     // Specific integer type
```

Warning

Type annotations are **required** for:

- Function parameters
- Function return types
- Struct fields
- Uninitialized variables

3.12 Structural Typing

Types are compatible based on structure, not name:

```
struct Point2D {
    x: float
    y: float
}

struct Vector2D {
    x: float
    y: float
}

fn magnitude(p: {x: float, y: float}) -> float {
    return math.sqrt(p.x * p.x + p.y * p.y)
}

// Both work because they have the same structure
point = Point2D{x: 3.0, y: 4.0}
vector = Vector2D{x: 3.0, y: 4.0}

io.println(magnitude(point))   // 5.0
io.println(magnitude(vector))  // 5.0
```

3.13 Type Grammar

```
type = primitive_type
     | array_type
     | map_type
     | tuple_type
```

```
     | option_type
     | function_type
     | generic_type
     | struct_type
     | identifier ;

primitive_type = "int" | "i8" | "i16" | "i32" | "i64"
               | "u8" | "u16" | "u32" | "u64"
               | "float" | "f32" | "f64"
               | "bool" | "string" ;

array_type = "[" "]" type ;

map_type = "[" type ":" type "]" ;

tuple_type = "(" type { "," type } ")" ;

option_type = type "?" ;

function_type = "fn" "(" [ type_list ] ")" [ "->" type ] ;

generic_type = identifier "<" type_list ">" ;

type_list = type { "," type } ;
```

Chapter 4

Expressions

Expressions produce values. Every expression in HAIRA has a type determined at compile time.

4.1 Primary Expressions

4.1.1 Literals

```
42                  // int literal
3.14                // float literal
"hello"             // string literal
true                // bool literal
nil                 // nil literal
```

4.1.2 Identifiers

```
x                   // Variable reference
user.name           // Field access
array[0]            // Index access
```

4.1.3 Parenthesized Expressions

```
(1 + 2) * 3         // Grouping for precedence
```

4.2 Arithmetic Expressions

```
a + b               // Addition
a - b               // Subtraction
a * b               // Multiplication
a / b               // Division
a % b               // Modulo (remainder)
-a                  // Unary negation
```

4.2.1 Integer Division

Integer division truncates toward zero:

```
7 / 3              // 2
-7 / 3             // -2
7 / -3             // -2
```

4.2.2 Modulo

The modulo operator returns the remainder:

```
7 % 3              // 1
-7 % 3             // -1
7 % -3             // 1
```

4.3 Comparison Expressions

```
a == b             // Equal
a != b             // Not equal
a < b              // Less than
a > b              // Greater than
a <= b             // Less or equal
a >= b             // Greater or equal
```

All comparison expressions produce `bool` values.

4.3.1 Equality

Equality (==) compares by value for primitives and by structural equality for composite types:

```
1 == 1                          // true
"hello" == "hello"              // true
[1, 2] == [1, 2]               // true
User{id: 1} == User{id: 1}   // true (structural)
```

4.4 Logical Expressions

```
a and b            // Logical AND (also &&)
a or b             // Logical OR (also ||)
not a              // Logical NOT (also !)
```

4.4.1 Short-Circuit Evaluation

Logical operators use short-circuit evaluation:

```
// 'b' is not evaluated if 'a' is false
a and b
```

```
3
4 // 'b' is not evaluated if 'a' is true
5 a or b
```

4.5 String Expressions

4.5.1 Concatenation

```
1 "Hello, " + "World!"          // "Hello, World!"
```

4.5.2 String Interpolation

```
1 name = "Alice"
2 age = 30
3 "Hello, ${name}! You are ${age} years old."
```

Expressions inside `${...}` are evaluated and converted to strings.

4.6 Array and Map Expressions

4.6.1 Array Literals

```
1 []                            // Empty array (type from context)
2 [1, 2, 3]                     // []int
3 ["a", "b", "c"]               // []string
```

4.6.2 Map Literals

```
1 [:]                           // Empty map (type from context)
2 ["a": 1, "b": 2]              // [string:int]
3 [1: "one", 2: "two"]          // [int:string]
```

4.6.3 Index Access

```
1 array[0]                      // First element
2 array[array.len - 1]          // Last element
3 map["key"]                    // Map lookup (returns T?)
```

Note

Array indexing with an out-of-bounds index causes a runtime panic. Map indexing returns an option type.

4.7 Field Access

```
user.name                       // Struct field
point.x                         // Struct field
tuple.0                         // Tuple element
```

4.8 Function Calls

```
func()                          // No arguments
func(a, b, c)                   // With arguments
obj.method()                    // Method call
obj.method(a, b)                // Method with arguments
```

4.8.1 Chained Calls

```
text.trim().to_upper().split(" ")
```

4.9 Pipe Expressions

The pipe operator passes the left operand as the first argument to the right function:

```
// These are equivalent:
result = f(g(h(x)))
result = x | h | g | f
```

4.9.1 Pipe with Arguments

When the right side has arguments, the left value becomes the first argument:

```
// These are equivalent:
result = string.split(text, ",")
result = text | string.split(",")
```

4.9.2 Complex Pipelines

```
import "io"
import "string"
import "array"

fn main() {
    data = "  apple, banana, cherry  "

    result = data
        | string.trim
        | string.split(", ")
        | array.map(string.to_upper)
```

```
        | array.filter(fn(s) { string.len(s) > 5 })
        | string.join(" | ")

    io.println(result)  // "BANANA | CHERRY"
}
```

4.10 Range Expressions

```
0..10           // 0 to 9 (exclusive end)
0..=10          // 0 to 10 (inclusive end)
```

Ranges are used primarily in **for** loops:

```
for i in 0..5 {
    io.println(i)  // 0, 1, 2, 3, 4
}

for i in 0..=5 {
    io.println(i)  // 0, 1, 2, 3, 4, 5
}
```

4.11 Conditional Expressions

4.11.1 If Expression

if can be used as an expression:

```
max = if a > b { a } else { b }

status = if score >= 90 {
    "A"
} else if score >= 80 {
    "B"
} else {
    "C"
}
```

Warning

When used as an expression, all branches must have the same type and **else** is required.

4.11.2 Match Expression

match expressions provide pattern matching:

```
result = match value {
    0 => "zero"
    1 => "one"
    n if n < 0 => "negative"
    _ => "other"
```

```
}
```

See Chapter 5 for complete pattern matching semantics.

4.12 Block Expressions

Blocks are expressions that evaluate to their last expression:

```
result = {
    x = compute_x()
    y = compute_y()
    x + y  // This is the value of the block
}
```

4.13 Struct Instantiation

```
user = User{
    id: 1,
    name: "Alice",
    email: "alice@example.com"
}

// With shorthand (field name matches variable)
name = "Bob"
email = "bob@example.com"
user = User{id: 2, name, email}
```

4.14 Closure Expressions

Anonymous functions (closures):

```
// Full syntax
add = fn(a: int, b: int) -> int {
    return a + b
}

// Short syntax for single expression
double = fn(x: int) -> int { x * 2 }

// Type inference for closures
numbers = [1, 2, 3]
doubled = array.map(numbers, fn(n) { n * 2 })
```

4.15 Option Expressions

4.15.1 Nil Coalescing

The **or** operator provides a default for optional values:

```
name = maybe_name or "Unknown"
```

```
value = get_value() or default_value
```

4.15.2 Optional Chaining

```
// Returns nil if any part is nil
street = user?.address?.street
```

4.16 Type Assertions

```
// Assert that value is a specific type
x = value as int

// Safe assertion (returns option)
x = value as? int
```

4.17 Operator Precedence

From highest to lowest:

1. Postfix: `() [] . ?.`
2. Unary: `! - not`
3. Multiplicative: `* / %`
4. Additive: `+ -`
5. Range: `.. ..=`
6. Pipe: `|`
7. Comparison: `< > <= >=`
8. Equality: `== !=`
9. Logical AND: `and &&`
10. Logical OR: `or ||`
11. Assignment: `= += -= *= /=`

4.18 Expression Grammar

```
expression = assignment ;

assignment = or_expr [ ("=" | "+=" | "-=" | "*=" | "/=")
    assignment ] ;

or_expr = and_expr { ("or" | "||") and_expr } ;

```

```
and_expr = equality { ("and" | "&&") equality } ;

equality = comparison { ("==" | "!=") comparison } ;

comparison = pipe { ("<" | ">" | "<=" | ">=") pipe } ;

pipe = range { "|" range } ;

range = additive [ (".." | "..=") additive ] ;

additive = multiplicative { ("+" | "-") multiplicative } ;

multiplicative = unary { ("*" | "/" | "%") unary } ;

unary = ("!" | "-" | "not") unary | postfix ;

postfix = primary { call | index | field } ;

call = "(" [ arguments ] ")" ;

index = "[" expression "]" ;

field = "." identifier ;

primary = identifier
        | literal
        | "(" expression ")"
        | array_literal
        | map_literal
        | struct_literal
        | closure
        | if_expr
        | match_expr
        | block ;

arguments = expression { "," expression } ;
```

Chapter 5

Statements

Statements perform actions. Unlike expressions, statements do not produce values.

5.1 Variable Declarations

5.1.1 With Inference

```
x = 42                    // Type inferred as int
name = "Haira"            // Type inferred as string
items = [1, 2, 3]         // Type inferred as []int
```

5.1.2 With Explicit Type

```
x: int = 42
name: string = "Haira"
count: i32 = 100
```

5.1.3 Multiple Declarations

```
a, b = 1, 2
x, y, z = compute()       // Destructuring tuple
```

5.1.4 Uninitialized Variables

```
x: int                    // Must be assigned before use
```

Warning

Using an uninitialized variable is a compile-time error.

5.2 Assignment Statements

```
x = 10                    // Simple assignment
x += 5                    // Add and assign (x = x + 5)
x -= 3                    // Subtract and assign
x *= 2                    // Multiply and assign
x /= 4                    // Divide and assign
```

5.2.1 Multiple Assignment

```
a, b = b, a               // Swap values
x, y = get_coordinates()
quotient, remainder = divide(17, 5)
```

5.2.2 Field Assignment

```
user.name = "Bob"
point.x = 10.0
```

5.2.3 Index Assignment

```
array[0] = 100
map["key"] = "value"
```

5.3 Expression Statements

Any expression can be used as a statement:

```
io.println("Hello")       // Function call
process(data)             // Function call, ignoring return
x + y                     // Valid but pointless
```

Note

The compiler may warn about expression statements that have no side effects.

5.4 If Statements

```
if condition {
    // executed if condition is true
}

if condition {
    // true branch
} else {
    // false branch
}
```

```

if condition1 {
    // first
} else if condition2 {
    // second
} else {
    // default
}
```

5.4.1 Condition Binding

```
// Bind and check in one statement
if value = get_optional_value(); value != nil {
    io.println(value)
}
```

5.5 For Statements

5.5.1 Range-Based For

```
// Exclusive range
for i in 0..10 {
    io.println(i)  // 0 through 9
}

// Inclusive range
for i in 0..=10 {
    io.println(i)  // 0 through 10
}
```

5.5.2 Collection Iteration

```
// Array iteration
for item in items {
    io.println(item)
}

// With index
for i, item in items {
    io.println("${i}: ${item}")
}

// Map iteration
for key, value in scores {
    io.println("${key}: ${value}")
}
```

5.5.3 String Iteration

```
for char in "hello" {
    io.println(char)  // Each UTF-8 character
}

for i, char in "hello" {
    io.println("${i}: ${char}")
}
```

5.6 While Statements

```
while condition {
    // body
}

// Infinite loop
while true {
    if should_stop() {
        break
    }
}
```

5.7 Match Statements

5.7.1 Basic Matching

```
match value {
    1 => io.println("one")
    2 => io.println("two")
    3 => io.println("three")
    _ => io.println("other")
}
```

5.7.2 Multiple Patterns

```
match value {
    1 | 2 | 3 => io.println("small")
    4 | 5 | 6 => io.println("medium")
    _ => io.println("large")
}
```

5.7.3 Range Patterns

```
match score {
    90..=100 => io.println("A")
    80..90 => io.println("B")
    70..80 => io.println("C")
    60..70 => io.println("D")
```

```
    _ => io.println("F")
}
```

5.7.4 Guard Clauses

```
match value {
    n if n < 0 => io.println("negative")
    n if n == 0 => io.println("zero")
    n if n > 0 => io.println("positive")
}
```

5.7.5 Destructuring Patterns

```
// Struct destructuring
match user {
    User{name: "admin", active: true} => io.println("Active
        admin")
    User{name: n, active: false} => io.println("Inactive: " + n)
    _ => io.println("Regular user")
}

// Tuple destructuring
match point {
    (0, 0) => io.println("origin")
    (x, 0) => io.println("on x-axis at " + to_string(x))
    (0, y) => io.println("on y-axis at " + to_string(y))
    (x, y) => io.println("at (" + to_string(x) + ", " +
        to_string(y) + ")")
}

// Enum destructuring
match result {
    Result.Ok(value) => io.println("Got: " + to_string(value))
    Result.Err(err) => io.println("Error: " + err.message)
}
```

5.7.6 Match Exhaustiveness

```
enum Direction { North, South, East, West }

match direction {
    Direction.North => go_north()
    Direction.South => go_south()
    Direction.East => go_east()
    Direction.West => go_west()
    // No _ needed: all cases covered
}
```

Note

The compiler verifies that match expressions are exhaustive. If not all cases are covered and there is no wildcard (_), the compiler produces an error.

5.8 Block Statements

```
{
    x = 10
    y = 20
    io.println(x + y)
}
```

Blocks create a new scope. Variables declared inside are not visible outside.

5.9 Return Statements

```
fn add(a: int, b: int) -> int {
    return a + b
}

fn early_return(x: int) -> int {
    if x < 0 {
        return 0
    }
    return x * 2
}

// Multiple returns
fn divide(a: int, b: int) -> (int, Error?) {
    if b == 0 {
        return 0, Error{message: "division by zero"}
    }
    return a / b, nil
}
```

5.9.1 Implicit Return

The last expression in a function is implicitly returned:

```
fn add(a: int, b: int) -> int {
    a + b  // No return keyword needed
}
```

5.10 Break Statements

```
for i in 0..100 {
    if i > 10 {
        break
```

```
    }
    io.println(i)
}

while true {
    if done() {
        break
    }
}
```

5.10.1 Labeled Break

```
outer: for i in 0..10 {
    for j in 0..10 {
        if i * j > 25 {
            break outer
        }
    }
}
```

5.11 Continue Statements

```
for i in 0..10 {
    if i % 2 == 0 {
        continue  // Skip even numbers
    }
    io.println(i)
}
```

5.11.1 Labeled Continue

```
outer: for i in 0..5 {
    for j in 0..5 {
        if j == 2 {
            continue outer
        }
        io.println("${i}, ${j}")
    }
}
```

5.12 Import Statements

Import statements must appear at the top of the file:

```
import "io"
import "json"
import db from "haira/postgres"
import { User, Post } from "models"
```

See Chapter 9 for complete import semantics.

5.13 Defer Statements

defer schedules a statement to run when the current scope exits:

```
fn read_file(path: string) -> (string, Error?) {
    file, err = fs.open(path)
    if err != nil {
        return "", err
    }
    defer fs.close(file)

    content, err = fs.read_all(file)
    if err != nil {
        return "", err  // file is closed here
    }

    return content, nil  // file is closed here too
}
```

5.13.1 Multiple Defers

Multiple **defer** statements execute in reverse order (LIFO):

```
fn example() {
    defer io.println("first")   // Runs third
    defer io.println("second")  // Runs second
    defer io.println("third")   // Runs first
}
// Output: third, second, first
```

5.14 Statement Grammar

```
statement = variable_decl
          | assignment
          | expression_stmt
          | if_stmt
          | for_stmt
          | while_stmt
          | match_stmt
          | return_stmt
          | break_stmt
          | continue_stmt
          | defer_stmt
          | block ;

variable_decl = identifier [ ":" type ] "=" expression
              | identifier_list "=" expression_list ;

assignment = lvalue assign_op expression ;

assign_op = "=" | "+=" | "-=" | "*=" | "/=" ;
```

```
20
21 lvalue = identifier | field_access | index_access ;
22
23 expression_stmt = expression ;
24
25 if_stmt = "if" [ simple_stmt ";" ] expression block
26           [ "else" ( if_stmt | block ) ] ;
27
28 for_stmt = "for" [ identifier "," ] identifier "in" expression
       block ;
29
30 while_stmt = "while" expression block ;
31
32 match_stmt = "match" expression "{" { match_arm } "}" ;
33
34 match_arm = pattern [ "if" expression ] "=>" ( expression |
       block ) ;
35
36 return_stmt = "return" [ expression_list ] ;
37
38 break_stmt = "break" [ identifier ] ;
39
40 continue_stmt = "continue" [ identifier ] ;
41
42 defer_stmt = "defer" statement ;
43
44 block = "{" { statement } "}" ;
```

Chapter 6

Functions

Functions are the primary unit of code organization in Haira. They are first-class values that can be passed as arguments, returned from other functions, and stored in variables.

6.1 Function Declarations

```
fn function_name(param1: Type1, param2: Type2) -> ReturnType {
    // body
}
```

6.1.1 Basic Examples

```
fn greet() {
    io.println("Hello!")
}

fn add(a: int, b: int) -> int {
    return a + b
}

fn is_positive(n: int) -> bool {
    return n > 0
}
```

6.1.2 No Return Type

Functions without a return type implicitly return `()`:

```
fn log_message(msg: string) {
    io.println("[LOG] " + msg)
}
```

6.2 Parameters

6.2.1 Required Parameters

All parameters must have explicit type annotations:

```
fn calculate(x: int, y: int, z: float) -> float {
    return (x + y) as float * z
}
```

6.2.2 Default Parameters

Parameters can have default values:

```
fn greet(name: string, greeting: string = "Hello") {
    io.println(greeting + ", " + name + "!")
}

greet("Alice")              // "Hello, Alice!"
greet("Bob", "Hi")          // "Hi, Bob!"
```

Note

Parameters with defaults must come after parameters without defaults.

6.2.3 Named Arguments

Functions can be called with named arguments:

```
fn create_user(name: string, email: string, age: int = 0) ->
   User {
    return User{name: name, email: email, age: age}
}

// Positional
create_user("Alice", "alice@example.com", 30)

// Named
create_user(name: "Bob", email: "bob@example.com")

// Mixed (positional first, then named)
create_user("Charlie", email: "charlie@example.com", age: 25)
```

6.2.4 Variadic Parameters

Functions can accept variable numbers of arguments:

```
fn sum(numbers: ...int) -> int {
    total = 0
    for n in numbers {
        total += n
    }
    return total
```

```
}

sum(1, 2, 3)          // 6
sum(1, 2, 3, 4, 5)    // 15
```

6.3 Return Values

6.3.1 Single Return

```
fn double(n: int) -> int {
    return n * 2
}
```

6.3.2 Multiple Returns

Functions can return multiple values using tuples:

```
fn divide(a: int, b: int) -> (int, int) {
    return a / b, a % b
}

quotient, remainder = divide(17, 5)   // 3, 2
```

6.3.3 Implicit Return

The last expression in a function body is implicitly returned:

```
fn add(a: int, b: int) -> int {
    a + b   // Implicit return
}

fn max(a: int, b: int) -> int {
    if a > b { a } else { b }   // Implicit return
}
```

6.3.4 Early Return

Use **return** for early exit:

```
fn factorial(n: int) -> int {
    if n <= 1 {
        return 1
    }
    return n * factorial(n - 1)
}
```

6.4 Function Types

Functions have types that can be used in type annotations:

```
// Function type: fn(int, int) -> int
type BinaryOp = fn(int, int) -> int

fn apply(op: BinaryOp, a: int, b: int) -> int {
    return op(a, b)
}

fn add(a: int, b: int) -> int { a + b }
fn mul(a: int, b: int) -> int { a * b }

apply(add, 2, 3)  // 5
apply(mul, 2, 3)  // 6
```

6.5 Closures

Closures are anonymous functions that can capture values from their environment:

```
fn make_counter() -> fn() -> int {
    count = 0
    return fn() -> int {
        count += 1
        return count
    }
}

counter = make_counter()
counter()  // 1
counter()  // 2
counter()  // 3
```

6.5.1 Closure Syntax

```
// Full syntax
fn(a: int, b: int) -> int {
    return a + b
}

// Short syntax (single expression)
fn(a: int, b: int) -> int { a + b }

// With type inference (in context)
numbers = [1, 2, 3]
doubled = array.map(numbers, fn(n) { n * 2 })
```

6.5.2 Capture Semantics

Closures capture variables by reference:

```
fn example() {
    values = []

```

```
    for i in 0..3 {
        // Captures 'i' by reference
        values = array.push(values, fn() { i })
    }

    // All closures see final value of i
    values[0]()  // 3
    values[1]()  // 3
    values[2]()  // 3
}
```

To capture by value, use explicit binding:

```
fn example() {
    values = []

    for i in 0..3 {
        captured = i  // Capture current value
        values = array.push(values, fn() { captured })
    }

    values[0]()  // 0
    values[1]()  // 1
    values[2]()  // 2
}
```

6.6 Methods

Methods are functions associated with a type:

```
struct Rectangle {
    width: float
    height: float
}

fn (r: Rectangle) area() -> float {
    return r.width * r.height
}

fn (r: Rectangle) perimeter() -> float {
    return 2.0 * (r.width + r.height)
}

fn (r: Rectangle) scale(factor: float) -> Rectangle {
    return Rectangle{
        width: r.width * factor,
        height: r.height * factor
    }
}

// Usage
rect = Rectangle{width: 10.0, height: 5.0}
rect.area()       // 50.0
rect.perimeter()  // 30.0
rect.scale(2.0)   // Rectangle{width: 20.0, height: 10.0}
```

6.6.1 Method Receiver

The receiver can be by value or by reference:

```
// By value (receiver is copied)
fn (p: Point) distance_to(other: Point) -> float {
    dx = p.x - other.x
    dy = p.y - other.y
    return math.sqrt(dx*dx + dy*dy)
}

// By mutable reference (can modify receiver)
fn (p: *Point) translate(dx: float, dy: float) {
    p.x += dx
    p.y += dy
}
```

6.7 Generic Functions

Functions can be parameterized over types:

```
fn identity<T>(value: T) -> T {
    return value
}

fn swap<T, U>(pair: (T, U)) -> (U, T) {
    return (pair.1, pair.0)
}

fn first<T>(items: []T) -> T? {
    if array.len(items) == 0 {
        return nil
    }
    return items[0]
}
```

6.7.1 Type Constraints

Generic types can be constrained:

```
constraint Comparable {
    fn compare(other: Self) -> int
}

fn max<T: Comparable>(a: T, b: T) -> T {
    if a.compare(b) > 0 {
        return a
    }
    return b
}

fn sort<T: Comparable>(items: []T) -> []T {
    // Sorting implementation
}
```

6.8 Higher-Order Functions

Functions that take or return other functions:

```
// Function as parameter
fn apply_twice(f: fn(int) -> int, x: int) -> int {
    return f(f(x))
}

fn double(n: int) -> int { n * 2 }

apply_twice(double, 5)  // 20

// Function as return value
fn make_multiplier(factor: int) -> fn(int) -> int {
    return fn(n: int) -> int {
        return n * factor
    }
}

triple = make_multiplier(3)
triple(7)  // 21
```

6.9 Recursion

Functions can call themselves:

```
fn factorial(n: int) -> int {
    if n <= 1 {
        return 1
    }
    return n * factorial(n - 1)
}

fn fibonacci(n: int) -> int {
    match n {
        0 => 0
        1 => 1
        _ => fibonacci(n - 1) + fibonacci(n - 2)
    }
}
```

6.9.1 Tail Call Optimization

HAIRA optimizes tail-recursive functions:

```
fn factorial_tail(n: int, acc: int = 1) -> int {
    if n <= 1 {
        return acc
    }
    return factorial_tail(n - 1, n * acc)  // Tail call
}
```

6.10 Function Overloading

Haira does not support function overloading. Each function name must be unique within its scope. Use different names or generic functions instead:

```
// Instead of overloading, use descriptive names
fn add_ints(a: int, b: int) -> int { a + b }
fn add_floats(a: float, b: float) -> float { a + b }

// Or use generics
fn add<T: Addable>(a: T, b: T) -> T { a + b }
```

6.11 The main Function

Every executable Haira program must have a `main` function:

```
fn main() {
    io.println("Hello, World!")
}

// With exit code
fn main() -> int {
    io.println("Hello, World!")
    return 0
}
```

6.12 Function Grammar

```
function_decl = "fn" [ receiver ] identifier [ type_params ]
                "(" [ params ] ")" [ "->" type ] block ;

receiver = "(" identifier ":" [ "*" ] type ")" ;

type_params = "<" type_param { "," type_param } ">" ;

type_param = identifier [ ":" constraint_list ] ;

constraint_list = identifier { "+" identifier } ;

params = param { "," param } ;

param = identifier ":" [ "..." ] type [ "=" expression ] ;

closure = "fn" "(" [ closure_params ] ")" [ "->" type ]
          ( block | expression ) ;

closure_params = closure_param { "," closure_param } ;

closure_param = identifier [ ":" type ] ;
```

Chapter 7

Error Handling

HAIRA uses explicit error handling inspired by Go. Errors are values, not exceptions. Functions that can fail return both a result and an optional error.

7.1 Philosophy

- **Errors are values** – Not special control flow
- **Explicit handling** – Callers must handle or propagate errors
- **No hidden control flow** – No exceptions or stack unwinding
- **Composable** – Errors can be wrapped and chained

7.2 The Error Type

The built-in `Error` type represents errors:

```
struct Error {
    message: string
    code: int?
    source: Error?
}
```

7.2.1 Creating Errors

```
// Simple error
err = Error{message: "something went wrong"}

// With error code
err = Error{message: "permission denied", code: 403}

// Wrapped error (error chaining)
err = Error{
    message: "failed to read config",
    source: original_error
}
```

7.3 Error Return Pattern

Functions that can fail return a tuple of `(result, Error?)`:

```
fn divide(a: int, b: int) -> (int, Error?) {
    if b == 0 {
        return 0, Error{message: "division by zero"}
    }
    return a / b, nil
}

fn read_file(path: string) -> (string, Error?) {
    // Implementation
}

fn parse_int(s: string) -> (int, Error?) {
    // Implementation
}
```

7.4 Handling Errors

7.4.1 Basic Error Checking

```
result, err = divide(10, 0)
if err != nil {
    io.println("Error: " + err.message)
    return
}
io.println("Result: " + to_string(result))
```

7.4.2 Propagating Errors

```
fn process_file(path: string) -> (Data, Error?) {
    content, err = read_file(path)
    if err != nil {
        return Data{}, err  // Propagate error
    }

    data, err = parse_data(content)
    if err != nil {
        return Data{}, err  // Propagate error
    }

    return data, nil
}
```

7.4.3 Wrapping Errors

Add context when propagating:

```
fn load_config(path: string) -> (Config, Error?) {
```

```
    content, err = read_file(path)
    if err != nil {
        return Config{}, Error{
            message: "failed to load config from " + path,
            source: err
        }
    }

    config, err = parse_config(content)
    if err != nil {
        return Config{}, Error{
            message: "invalid config format",
            source: err
        }
    }

    return config, nil
}
```

7.5 Error Checking Patterns

7.5.1 Guard Pattern

Check errors early and return:

```
fn process(input: string) -> (Output, Error?) {
    // Guard: validate input
    if string.len(input) == 0 {
        return Output{}, Error{message: "input cannot be empty"}
    }

    // Guard: check prerequisites
    ready, err = check_ready()
    if err != nil {
        return Output{}, err
    }
    if not ready {
        return Output{}, Error{message: "system not ready"}
    }

    // Main logic
    result = do_processing(input)
    return result, nil
}
```

7.5.2 Defer for Cleanup

Use **defer** to ensure cleanup happens even on error:

```
fn with_file(path: string) -> (string, Error?) {
    file, err = fs.open(path)
    if err != nil {
        return "", err
    }
    defer fs.close(file)  // Always closes, even on error
```

```

    content, err = fs.read_all(file)
    if err != nil {
        return "", err  // file is closed by defer
    }

    return content, nil  // file is closed by defer
}
```

7.6 Error Utilities

7.6.1 Checking Error Types

```
fn is_not_found(err: Error?) -> bool {
    if err == nil {
        return false
    }
    return err.code == 404 or string.contains(err.message, "not
        found")
}

fn is_permission_denied(err: Error?) -> bool {
    if err == nil {
        return false
    }
    return err.code == 403
}
```

7.6.2 Error Chain Traversal

```
fn root_cause(err: Error?) -> Error? {
    if err == nil {
        return nil
    }
    current = err
    while current.source != nil {
        current = current.source
    }
    return current
}

fn error_chain(err: Error?) -> []Error {
    errors = []
    current = err
    while current != nil {
        errors = array.push(errors, current)
        current = current.source
    }
    return errors
}
```

7.7 Option Types and Errors

For operations that may not find a value (not an error condition), use option types:

```
// Use option when "not found" is normal
fn find_user(id: int) -> User? {
    // Returns nil if not found
}

// Use error when "not found" is exceptional
fn get_user(id: int) -> (User, Error?) {
    // Returns error if not found
}
```

7.7.1 Converting Between Options and Errors

```
// Option to error
fn require_user(id: int) -> (User, Error?) {
    user = find_user(id)
    if user == nil {
        return User{}, Error{message: "user not found", code:
            404}
    }
    return user, nil
}

// Error to option (ignoring error)
fn try_parse_int(s: string) -> int? {
    result, err = parse_int(s)
    if err != nil {
        return nil
    }
    return result
}
```

7.8 Panic and Recovery

For truly unrecoverable situations, use `panic`:

```
fn assert(condition: bool, message: string) {
    if not condition {
        panic(message)
    }
}

fn unwrap<T>(opt: T?, message: string) -> T {
    if opt == nil {
        panic(message)
    }
    return opt
}
```

Warning

`panic` terminates the program. Use it only for programming errors, not for expected failure conditions. Prefer returning errors for all recoverable situations.

7.8.1 When to Use Panic

- Programming bugs (assertions)
- Invariant violations
- Unrecoverable initialization failures
- Unreachable code paths

7.8.2 When to Use Errors

- File not found
- Network failures
- Invalid user input
- Resource exhaustion
- Any expected failure condition

7.9 Best Practices

7.9.1 Return Errors, Don't Log and Continue

```
// Bad: logs and continues with invalid state
fn process(data: Data) -> Result {
    result, err = validate(data)
    if err != nil {
        log.error("validation failed: " + err.message)
        // Continues with unvalidated data!
    }
    // ...
}

// Good: returns error to caller
fn process(data: Data) -> (Result, Error?) {
    result, err = validate(data)
    if err != nil {
        return Result{}, Error{
            message: "validation failed",
            source: err
        }
    }
    // ...
}
```

7.9.2 Add Context When Wrapping

```
// Bad: just propagates
if err != nil {
    return Result{}, err
}

// Good: adds context
if err != nil {
    return Result{}, Error{
        message: "failed to process user " + to_string(user_id),
        source: err
    }
}
```

7.9.3 Handle Errors at the Right Level

```
// Low-level: specific error
fn read_config_file(path: string) -> ([]byte, Error?) {
    // Returns "file not found" or "permission denied"
}

// Mid-level: wrapped with context
fn load_config(path: string) -> (Config, Error?) {
    data, err = read_config_file(path)
    if err != nil {
        return Config{}, Error{
            message: "could not load config",
            source: err
        }
    }
    // Parse...
}

// High-level: user-friendly handling
fn main() {
    config, err = load_config("config.json")
    if err != nil {
        io.println("Error: " + err.message)
        // Could show root_cause for debugging
        os.exit(1)
    }
    // Continue...
}
```

7.10 Error Handling Grammar

Error handling uses standard language constructs:

```
error_return = "(" type "," "Error?" ")" ;

error_check = "if" identifier "!=" "nil" block ;

```

```
error_create = "Error" "{"
               "message" ":" expression
               [ "," "code" ":" expression ]
               [ "," "source" ":" expression ]
               "}" ;
```

Part II

Advanced Features

Chapter 8

Concurrency

HAIRA provides Go-style concurrency primitives: lightweight spawned tasks and channels for communication. The philosophy is "share memory by communicating" rather than "communicate by sharing memory."

8.1 Concurrency Model

- **Spawned tasks** – Lightweight concurrent execution units
- **Channels** – Typed conduits for communication
- **Select** – Multiplexing channel operations
- **No shared mutable state** – Communication via channels

8.2 Spawn

The **spawn** keyword starts a new concurrent task:

```
import "io"
import "time"

fn main() {
    spawn {
        time.sleep(1000)
        io.println("Hello from spawned task!")
    }

    io.println("Main continues immediately")
    time.sleep(2000)  // Wait for spawned task
}
```

8.2.1 Spawning Functions

```
fn worker(id: int) {
    io.println("Worker " + to_string(id) + " started")
    time.sleep(1000)
    io.println("Worker " + to_string(id) + " done")
}

```

```
fn main() {
    for i in 0..5 {
        spawn worker(i)
    }
    time.sleep(3000)  // Wait for all workers
}
```

8.2.2 Spawn Returns Handle

```
fn compute() -> int {
    time.sleep(1000)
    return 42
}

fn main() {
    handle = spawn compute()

    // Do other work...

    result = await handle  // Wait for result
    io.println("Result: " + to_string(result))
}
```

8.3 Channels

Channels are typed conduits for passing values between tasks:

```
// Create a channel
ch = chan<int>()            // Unbuffered channel
ch = chan<int>(10)          // Buffered channel (capacity 10)
ch = chan<string>()         // String channel
```

8.3.1 Sending and Receiving

```
ch = chan<int>()

// Send (blocks until received for unbuffered)
ch <- 42

// Receive (blocks until value available)
value = <-ch
```

8.3.2 Basic Channel Example

```
import "io"

fn main() {
    ch = chan<string>()

```

```
    spawn {
        ch <- "Hello from spawned task!"
    }

    message = <-ch
    io.println(message)
}
```

8.3.3 Buffered Channels

```
ch = chan<int>(3)  // Buffer size 3

// These don't block (buffer has space)
ch <- 1
ch <- 2
ch <- 3

// This would block (buffer full)
// ch <- 4

// Receive
io.println(<-ch)  // 1
io.println(<-ch)  // 2
io.println(<-ch)  // 3
```

8.3.4 Closing Channels

```
ch = chan<int>(10)

spawn {
    for i in 0..10 {
        ch <- i
    }
    close(ch)  // Signal no more values
}

// Iterate until channel closed
for value in ch {
    io.println(value)
}

io.println("Channel closed, loop exited")
```

8.3.5 Checking Channel State

```
// Receive with closed check
value, ok = <-ch
if not ok {
    io.println("Channel closed")
}

```

```
// Non-blocking receive
value, ok = ch.try_receive()
if ok {
    io.println("Got: " + to_string(value))
} else {
    io.println("No value available")
}

// Non-blocking send
ok = ch.try_send(42)
if not ok {
    io.println("Channel full or closed")
}
```

8.4 Select

The **select** statement waits on multiple channel operations:

```
import "io"
import "time"

fn main() {
    ch1 = chan<string>()
    ch2 = chan<string>()

    spawn {
        time.sleep(1000)
        ch1 <- "from channel 1"
    }

    spawn {
        time.sleep(500)
        ch2 <- "from channel 2"
    }

    // Wait for first available
    select {
        msg = <-ch1 => io.println("Received: " + msg)
        msg = <-ch2 => io.println("Received: " + msg)
    }
}
```

8.4.1 Select with Default

Non-blocking select:

```
select {
    value = <-ch => io.println("Got: " + to_string(value))
    default => io.println("No value ready")
}
```

8.4.2 Select with Timeout

```
timeout = time.after(5000)  // 5 seconds

select {
    value = <-ch => io.println("Got: " + to_string(value))
    <-timeout => io.println("Timed out!")
}
```

8.4.3 Select in Loop

```
fn worker(jobs: chan<Job>, results: chan<Result>, done:
    chan<bool>) {
    while true {
        select {
            job = <-jobs => {
                result = process(job)
                results <- result
            }
            <-done => {
                io.println("Worker shutting down")
                return
            }
        }
    }
}
```

8.5 Common Patterns

8.5.1 Worker Pool

```
fn worker(id: int, jobs: chan<int>, results: chan<int>) {
    for job in jobs {
        io.println("Worker " + to_string(id) + " processing " +
            to_string(job))
        time.sleep(100)
        results <- job * 2
    }
}

fn main() {
    num_jobs = 10
    num_workers = 3

    jobs = chan<int>(num_jobs)
    results = chan<int>(num_jobs)

    // Start workers
    for i in 0..num_workers {
        spawn worker(i, jobs, results)
    }

    // Send jobs
    for j in 0..num_jobs {
```

```
        jobs <- j
    }
    close(jobs)

    // Collect results
    for _ in 0..num_jobs {
        result = <-results
        io.println("Result: " + to_string(result))
    }
}
```

8.5.2 Fan-Out, Fan-In

```
fn producer(out: chan<int>) {
    for i in 0..100 {
        out <- i
    }
    close(out)
}

fn worker(in: chan<int>, out: chan<int>) {
    for value in in {
        out <- value * value
    }
}

fn main() {
    input = chan<int>(100)
    output = chan<int>(100)

    // Producer
    spawn producer(input)

    // Fan-out to multiple workers
    for _ in 0..4 {
        spawn worker(input, output)
    }

    // Fan-in (collect results)
    spawn {
        time.sleep(1000)
        close(output)
    }

    for result in output {
        io.println(result)
    }
}
```

8.5.3 Pipeline

```
fn generate(out: chan<int>) {
    for i in 2..100 {
```

```
        out <- i
    }
    close(out)
}

fn filter(in: chan<int>, out: chan<int>, prime: int) {
    for n in in {
        if n % prime != 0 {
            out <- n
        }
    }
    close(out)
}

fn sieve() -> chan<int> {
    primes = chan<int>()

    spawn {
        ch = chan<int>(100)
        spawn generate(ch)

        while true {
            prime, ok = <-ch
            if not ok {
                break
            }
            primes <- prime

            next = chan<int>(100)
            spawn filter(ch, next, prime)
            ch = next
        }
        close(primes)
    }

    return primes
}

fn main() {
    for prime in sieve() {
        io.println(prime)
        if prime > 50 {
            break
        }
    }
}
```

8.5.4 Semaphore

```
struct Semaphore {
    ch: chan<bool>
}

fn new_semaphore(n: int) -> Semaphore {
    sem = Semaphore{ch: chan<bool>(n)}
    for _ in 0..n {
```

```
        sem.ch <- true
    }
    return sem
}

fn (s: Semaphore) acquire() {
    <-s.ch
}

fn (s: Semaphore) release() {
    s.ch <- true
}

// Usage: limit concurrent operations
fn main() {
    sem = new_semaphore(3)  // Max 3 concurrent

    for i in 0..10 {
        spawn {
            sem.acquire()
            defer sem.release()

            io.println("Task " + to_string(i) + " running")
            time.sleep(1000)
        }
    }

    time.sleep(5000)
}
```

8.6 Channel Types

8.6.1 Directional Channels

Functions can restrict channel direction:

```
// Send-only channel
fn producer(out: chan<-int) {
    out <- 42
    // <-out  // Error: cannot receive from send-only channel
}

// Receive-only channel
fn consumer(in: <-chan<int>) {
    value = <-in
    // in <- 42  // Error: cannot send to receive-only channel
}

fn main() {
    ch = chan<int>()
    spawn producer(ch)    // Converts to chan<-int
    spawn consumer(ch)    // Converts to <-chan<int>
}
```

8.7 Synchronization

8.7.1 WaitGroup

Wait for multiple tasks to complete:

```
import "sync"

fn main() {
    wg = sync.WaitGroup{}

    for i in 0..5 {
        wg.add(1)
        spawn {
            defer wg.done()
            io.println("Task " + to_string(i))
            time.sleep(1000)
        }
    }

    wg.wait()  // Block until all done
    io.println("All tasks completed")
}
```

8.7.2 Mutex

For rare cases requiring shared state:

```
import "sync"

struct Counter {
    mu: sync.Mutex
    value: int
}

fn (c: *Counter) increment() {
    c.mu.lock()
    defer c.mu.unlock()
    c.value += 1
}

fn (c: *Counter) get() -> int {
    c.mu.lock()
    defer c.mu.unlock()
    return c.value
}
```

Warning

Prefer channels over mutexes. Mutexes are provided for interoperability and performance-critical sections, but channel-based designs are generally safer and more idiomatic.

8.8 Concurrency Grammar

```
spawn_expr = "spawn" ( block | function_call ) ;

channel_type = "chan" "<" type ">"
             | "chan<-" type          (* send-only *)
             | "<-chan<" type ">"     (* receive-only *) ;

channel_create = "chan" "<" type ">" "(" [ expression ] ")" ;

send_stmt = expression "<-" expression ;

receive_expr = "<-" expression ;

select_stmt = "select" "{" { select_case } [ default_case ] "}" ;

select_case = pattern "=" receive_expr "=>" ( expression | block )
            | send_stmt "=>" ( expression | block ) ;

default_case = "default" "=>" ( expression | block ) ;
```

Chapter 9

Modules and Imports

HAIRA uses explicit imports for all dependencies. Every external symbol must be imported before use.

9.1 Module System Overview

- **Explicit imports** – All dependencies declared at file top
- **No implicit globals** – Everything must be imported
- **Simple resolution** – Standard library, project-local, external
- **Namespaced access** – `module.function()` syntax

9.2 Import Syntax

9.2.1 Basic Import

```
import "io"
import "json"
import "http"

fn main() {
    io.println("Hello")
}
```

9.2.2 Aliased Import

```
import fmt from "io"
import db from "haira/postgres"

fn main() {
    fmt.println("Hello")
    db.connect("localhost:5432")
}
```

9.2.3 Selective Import

Import specific symbols:

```
import { println, readln } from "io"
import { User, Post } from "models"

fn main() {
    println("Enter name:")
    name = readln()
    println("Hello, " + name)
}
```

9.2.4 Glob Import

Import all exports (use sparingly):

```
import * from "math"

fn main() {
    result = sqrt(pow(3.0, 2.0) + pow(4.0, 2.0))
    io.println(result)  // 5.0
}
```

Warning

Glob imports can cause naming conflicts and make code harder to read. Prefer explicit imports.

9.3 Module Resolution

Imports are resolved in this order:

1. **Standard library** – Built-in modules (`"io"`, `"json"`, etc.)
2. **Project-local** – Relative to project root (`"models/user"`)
3. **External packages** – From package registry (`"github.com/..."`)

9.3.1 Standard Library

```
import "io"           // Standard I/O
import "string"       // String manipulation
import "math"         // Mathematical functions
import "json"         // JSON encoding/decoding
import "http"         // HTTP client/server
import "fs"           // File system
import "os"           // Operating system
import "time"         // Time and duration
import "crypto"       // Cryptography
import "regex"        // Regular expressions
```

9.3.2 Project-Local Imports

```
// Project structure:
// myapp/
//   main.haira
//   models/
//     user.haira
//     post.haira
//   utils/
//     helpers.haira

// In main.haira:
import "models/user"
import "models/post"
import "utils/helpers"

// In models/post.haira:
import "models/user"    // Absolute from project root
```

9.3.3 External Packages

```
import "github.com/haira-libs/aws"
import "github.com/haira-libs/postgres"
import pg from "github.com/haira-libs/postgres"
```

External packages are declared in `haira.toml`:

```
[dependencies]
aws = "github.com/haira-libs/aws@1.0.0"
postgres = "github.com/haira-libs/postgres@2.1.0"
```

9.4 Module Definitions

9.4.1 File as Module

Each `.haira` file is a module. The file name determines the module name:

```
// File: utils/helpers.haira
// Module: utils/helpers

fn format_name(first: string, last: string) -> string {
    return first + " " + last
}

fn validate_email(email: string) -> bool {
    return string.contains(email, "@")
}
```

9.4.2 Exporting

By default, all top-level declarations are exported. Use underscore prefix for private:

```
// Public (exported)
fn public_function() { }
struct PublicType { }

// Private (not exported)
fn _private_function() { }
struct _PrivateType { }

// Private helper
_cache = [:]
```

9.4.3 Package Initialization

Optional `init()` function runs when module is first imported:

```
// database.haira

_pool: ConnectionPool?

fn init() {
    _pool = create_pool()
    io.println("Database module initialized")
}

fn get_connection() -> (Connection, Error?) {
    if _pool == nil {
        return Connection{}, Error{message: "not initialized"}
    }
    return _pool.get(), nil
}
```

Note

`init()` functions are called in dependency order. Circular dependencies are a compile-time error.

9.5 Package Structure

9.5.1 Single-File Package

```
myapp/
  main.haira
  haira.toml
```

9.5.2 Multi-File Package

```
myapp/
  main.haira
  models/
    user.haira
```

```
    post.haira
    mod.haira       # Package entry point (optional)
  services/
    auth.haira
    api.haira
  utils/
    helpers.haira
  haira.toml
```

9.5.3 The mod.haira File

Re-exports from a directory:

```haira
// models/mod.haira
import { User } from "models/user"
import { Post } from "models/post"

// Re-export
export { User, Post }
```

Now consumers can import from the directory:

```haira
import { User, Post } from "models"
```

9.6 Import Order

Imports should be organized in groups:

```haira
// 1. Standard library
import "io"
import "json"
import "http"

// 2. External packages
import "github.com/haira-libs/aws"

// 3. Project-local
import "models/user"
import "services/auth"
```

9.7 Circular Dependencies

Circular imports are not allowed:

```haira
// a.haira
import "b"  // Error if b imports a

// b.haira
import "a"  // Creates circular dependency
```

Solution: Extract shared code to a third module:

```
// shared.haira
struct Common { }

// a.haira
import "shared"

// b.haira
import "shared"
```

9.8 Conditional Imports

Import based on build configuration:

```
#[cfg(os = "windows")]
import "windows/api"

#[cfg(os = "linux")]
import "linux/api"

#[cfg(os = "macos")]
import "macos/api"
```

9.9 The haira.toml File

Project configuration:

```
[package]
name = "myapp"
version = "1.0.0"
authors = ["Your Name <you@example.com>"]

[dependencies]
aws = "github.com/haira-libs/aws@1.0.0"
postgres = "github.com/haira-libs/postgres@2.1.0"

[dev-dependencies]
testing = "github.com/haira-libs/testing@1.0.0"

[build]
target = "native"
optimization = "release"

[ai]
backend = "llama.cpp"
model_path = "~/.haira/models/codellama-7b.gguf"
```

9.10 Module Grammar

```
import_statement = "import" import_spec ;

```

```
import_spec = string_literal                              (* basic *)
            | identifier "from" string_literal            (* aliased *)
            | "{" identifier_list "}" "from" string_literal  (* selective *)
            | "*" "from" string_literal ;                 (* glob *)

export_statement = "export" "{" identifier_list "}" ;

identifier_list = identifier { "," identifier } ;

module = { import_statement } { export_statement } { top_level } ;
```

Chapter 10

Standard Library

The HAIRA standard library provides essential functionality for common programming tasks. All modules must be explicitly imported.

10.1 Core Modules

Module	Description
`io`	Input/output operations
`string`	String manipulation
`array`	Array operations
`map`	Map/dictionary operations
`math`	Mathematical functions
`conv`	Type conversions

Table 10.1: Core modules

10.2 io Module

Basic input/output operations.

```
import "io"

// Output
io.print("Hello")            // No newline
io.println("Hello")          // With newline
io.printf("Value: %d\n", 42)  // Formatted

// Input
line = io.readln()           // Read line from stdin

// Errors
io.eprintln("Error!")        // Print to stderr
```

10.2.1 io Functions

```
fn print(s: string)
```

```
fn println(s: string)
fn printf(format: string, args: ...any)
fn readln() -> string
fn eprintln(s: string)
fn eprintf(format: string, args: ...any)
```

10.3 string Module

String manipulation functions.

```
import "string"

s = "  Hello, World!  "

// Basic operations
string.len(s)                    // 18
string.is_empty("")              // true

// Trimming
string.trim(s)                   // "Hello, World!"
string.trim_left(s)              // "Hello, World!  "
string.trim_right(s)             // "  Hello, World!"

// Case
string.to_upper(s)               // "  HELLO, WORLD!  "
string.to_lower(s)               // "  hello, world!  "

// Search
string.contains(s, "World")      // true
string.starts_with(s, "  H")     // true
string.ends_with(s, "!  ")       // true
string.index_of(s, "o")          // 4
string.last_index_of(s, "o")     // 8

// Split/Join
string.split("a,b,c", ",")       // ["a", "b", "c"]
string.join(["a", "b"], "-")     // "a-b"

// Substring
string.substring(s, 2, 7)        // "Hello"
string.char_at(s, 2)             // "H"

// Replace
string.replace(s, "World", "Haira")  // "  Hello, Haira!  "
string.replace_all(s, " ", "_")      // "__Hello,_World!__"

// Repeat
string.repeat("ab", 3)           // "ababab"
```

10.3.1 string Functions

```
fn len(s: string) -> int
fn is_empty(s: string) -> bool
fn trim(s: string) -> string
```

```
fn trim_left(s: string) -> string
fn trim_right(s: string) -> string
fn to_upper(s: string) -> string
fn to_lower(s: string) -> string
fn contains(s: string, sub: string) -> bool
fn starts_with(s: string, prefix: string) -> bool
fn ends_with(s: string, suffix: string) -> bool
fn index_of(s: string, sub: string) -> int
fn last_index_of(s: string, sub: string) -> int
fn split(s: string, sep: string) -> []string
fn join(parts: []string, sep: string) -> string
fn substring(s: string, start: int, end: int) -> string
fn char_at(s: string, index: int) -> string
fn replace(s: string, old: string, new: string) -> string
fn replace_all(s: string, old: string, new: string) -> string
fn repeat(s: string, n: int) -> string
```

10.4 array Module

Array operations.

```
import "array"

arr = [1, 2, 3, 4, 5]

// Basic
array.len(arr)                    // 5
array.is_empty([])                // true

// Access
array.first(arr)                  // 1 (or nil if empty)
array.last(arr)                   // 5 (or nil if empty)
array.get(arr, 2)                 // 3 (or nil if out of bounds)

// Modification (returns new array)
array.push(arr, 6)                // [1, 2, 3, 4, 5, 6]
array.pop(arr)                    // ([1, 2, 3, 4], 5)
array.insert(arr, 0, 0)           // [0, 1, 2, 3, 4, 5]
array.remove(arr, 2)              // [1, 2, 4, 5]

// Slicing
array.slice(arr, 1, 4)            // [2, 3, 4]
array.take(arr, 3)                // [1, 2, 3]
array.drop(arr, 2)                // [3, 4, 5]

// Search
array.contains(arr, 3)            // true
array.index_of(arr, 3)            // 2
array.find(arr, fn(x) { x > 3 }) // 4

// Transform
array.map(arr, fn(x) { x * 2 })  // [2, 4, 6, 8, 10]
array.filter(arr, fn(x) { x > 2 })  // [3, 4, 5]
array.reduce(arr, 0, fn(acc, x) { acc + x })  // 15

// Sort
```

```
array.sort(arr)                    // [1, 2, 3, 4, 5]
array.sort_by(arr, fn(a, b) { b - a })  // [5, 4, 3, 2, 1]

// Other
array.reverse(arr)                 // [5, 4, 3, 2, 1]
array.concat(arr, [6, 7])          // [1, 2, 3, 4, 5, 6, 7]
array.flatten([[1, 2], [3, 4]])    // [1, 2, 3, 4]
array.unique([1, 2, 2, 3])         // [1, 2, 3]
```

10.5 map Module

Map/dictionary operations.

```
import "map"

m = ["a": 1, "b": 2, "c": 3]

// Basic
map.len(m)                         // 3
map.is_empty(m)                    // false

// Access
map.get(m, "a")                    // 1 (or nil if not found)
map.has(m, "a")                    // true

// Modification (returns new map)
map.set(m, "d", 4)                 // ["a": 1, "b": 2, "c": 3,
    "d": 4]
map.remove(m, "b")                 // ["a": 1, "c": 3]

// Iteration helpers
map.keys(m)                        // ["a", "b", "c"]
map.values(m)                      // [1, 2, 3]
map.entries(m)                     // [("a", 1), ("b", 2), ("c",
    3)]

// Transform
map.map_values(m, fn(v) { v * 2 })  // ["a": 2, "b": 4, "c": 6]
map.filter(m, fn(k, v) { v > 1 })   // ["b": 2, "c": 3]

// Merge
map.merge(m, ["d": 4])             // ["a": 1, "b": 2, "c": 3,
    "d": 4]
```

10.6 math Module

Mathematical functions.

```
import "math"

// Constants
math.PI                            // 3.14159265358979
math.E                             // 2.71828182845905

```

```
// Basic
math.abs(-5)                    // 5
math.min(3, 7)                  // 3
math.max(3, 7)                  // 7
math.clamp(5, 0, 10)            // 5
math.clamp(-5, 0, 10)           // 0

// Rounding
math.floor(3.7)                 // 3.0
math.ceil(3.2)                  // 4.0
math.round(3.5)                 // 4.0
math.trunc(3.7)                 // 3.0

// Power/Root
math.pow(2.0, 10.0)             // 1024.0
math.sqrt(16.0)                 // 4.0
math.cbrt(27.0)                 // 3.0

// Exponential/Logarithm
math.exp(1.0)                   // 2.718...
math.log(math.E)                // 1.0
math.log10(100.0)               // 2.0
math.log2(8.0)                  // 3.0

// Trigonometry
math.sin(0.0)                   // 0.0
math.cos(0.0)                   // 1.0
math.tan(0.0)                   // 0.0
math.asin(0.0)                  // 0.0
math.acos(1.0)                  // 0.0
math.atan(0.0)                  // 0.0
math.atan2(1.0, 1.0)            // 0.785... (PI/4)

// Random
math.random()                   // Random float 0.0..1.0
math.random_int(1, 100)         // Random int 1..100
```

10.7 conv Module

Type conversion utilities.

```
import "conv"

// To string
conv.int_to_string(42)          // "42"
conv.float_to_string(3.14)      // "3.14"
conv.bool_to_string(true)       // "true"

// From string
conv.string_to_int("42")        // (42, nil)
conv.string_to_int("abc")       // (0, Error)
conv.string_to_float("3.14")    // (3.14, nil)
conv.string_to_bool("true")     // (true, nil)

// Number conversions
conv.int_to_float(42)           // 42.0
```

```
conv.float_to_int(3.7)              // 3 (truncates)

// Base conversions
conv.int_to_hex(255)                // "ff"
conv.int_to_binary(10)              // "1010"
conv.int_to_octal(8)                // "10"
conv.hex_to_int("ff")               // (255, nil)
```

10.8 Extended Modules

10.8.1 fs Module

File system operations.

```
import "fs"

// Read/Write
content, err = fs.read_file("file.txt")
err = fs.write_file("file.txt", "content")
err = fs.append_file("file.txt", "more")

// File operations
exists = fs.exists("file.txt")
err = fs.remove("file.txt")
err = fs.rename("old.txt", "new.txt")
err = fs.copy("src.txt", "dst.txt")

// Directory operations
err = fs.mkdir("dir")
err = fs.mkdir_all("path/to/dir")
entries, err = fs.read_dir("dir")
err = fs.remove_all("dir")

// File info
info, err = fs.stat("file.txt")
info.size                           // Size in bytes
info.is_dir                         // Is directory?
info.modified                       // Modification time
```

10.8.2 os Module

Operating system interface.

```
import "os"

// Environment
os.getenv("HOME")                   // Get env variable
os.setenv("KEY", "value")           // Set env variable
os.environ()                        // All env variables

// Process
os.args                             // Command line arguments
os.exit(0)                          // Exit with code
os.getpid()                         // Process ID

```

```
// Execution
output, err = os.exec("ls", ["-la"])
```

10.8.3 time Module

Time and duration.

```
import "time"

// Current time
now = time.now()
now.year                          // 2024
now.month                         // 1-12
now.day                           // 1-31
now.hour                          // 0-23
now.minute                        // 0-59
now.second                        // 0-59

// Formatting
time.format(now, "2006-01-02")    // "2024-03-15"

// Parsing
t, err = time.parse("2024-03-15", "2006-01-02")

// Duration
time.sleep(1000)                  // Sleep 1 second (ms)
duration = time.since(start)      // Time since start

// Timers
timer = time.after(5000)          // Channel that fires after 5s
ticker = time.tick(1000)          // Channel that fires every 1s
```

10.8.4 json Module

JSON encoding/decoding.

```
import "json"

struct User {
    name: string
    age: int
}

// Encoding
user = User{name: "Alice", age: 30}
data, err = json.encode(user)     // '{"name":"Alice","age":30}'

// Decoding
user, err = json.decode<User>(data)

// Pretty printing
data, err = json.encode_pretty(user)

// Raw JSON
value, err = json.parse('{"key": "value"}')
```

```
value["key"]                        // "value"
```

10.8.5 http Module

HTTP client and server.

```
import "http"

// GET request
response, err = http.get("https://api.example.com/users")
response.status                     // 200
response.body                       // Response body

// POST request
response, err = http.post(
    "https://api.example.com/users",
    ["Content-Type": "application/json"],
    '{"name": "Alice"}'
)

// Server
server = http.Server{port: 8080}

server.handle("/", fn(req: http.Request) -> http.Response {
    return http.Response{
        status: 200,
        body: "Hello, World!"
    }
})

server.listen()
```

10.8.6 regex Module

Regular expressions.

```
import "regex"

// Compile pattern
pattern, err = regex.compile(r"\d+")

// Match
regex.is_match(pattern, "abc123")  // true

// Find
match = regex.find(pattern, "abc123def")  // "123"
matches = regex.find_all(pattern, "a1b2c3")  // ["1", "2", "3"]

// Replace
regex.replace(pattern, "a1b2", "X")      // "aXb2"
regex.replace_all(pattern, "a1b2", "X") // "aXbX"

// Capture groups
pattern, _ = regex.compile(r"(\w+)@(\w+)")
captures = regex.captures(pattern, "alice@example")
```

```
// captures[0] = "alice@example"
// captures[1] = "alice"
// captures[2] = "example"
```

10.8.7 crypto Module

Cryptographic functions.

```
import "crypto"

// Hashing
crypto.md5("hello")                // Hash as hex string
crypto.sha256("hello")
crypto.sha512("hello")

// HMAC
crypto.hmac_sha256("message", "key")

// Random
crypto.random_bytes(32)            // 32 random bytes

// Encoding
crypto.base64_encode(bytes)
crypto.base64_decode(string)
crypto.hex_encode(bytes)
crypto.hex_decode(string)
```

10.8.8 net Module

Low-level networking.

```
import "net"

// TCP client
conn, err = net.dial("tcp", "example.com:80")
conn.write("GET / HTTP/1.1\r\n\r\n")
data, err = conn.read(1024)
conn.close()

// TCP server
listener, err = net.listen("tcp", ":8080")
while true {
    conn, err = listener.accept()
    spawn handle_connection(conn)
}
```

10.9 Testing Module

```
import "testing"

fn test_addition(t: testing.T) {
    result = add(2, 3)
```

```
    t.assert_eq(result, 5)
}

fn test_division(t: testing.T) {
    result, err = divide(10, 2)
    t.assert_nil(err)
    t.assert_eq(result, 5)

    _, err = divide(10, 0)
    t.assert_not_nil(err)
}
```

Run tests with:

```
$ haira test
```

Chapter 11

Advanced Features

This chapter covers advanced language features that build upon the core language.

11.1 Generics

Generics allow functions and types to be parameterized over types.

11.1.1 Generic Functions

```
fn identity<T>(value: T) -> T {
    return value
}

fn swap<T, U>(pair: (T, U)) -> (U, T) {
    return (pair.1, pair.0)
}

fn first<T>(items: []T) -> T? {
    if array.len(items) == 0 {
        return nil
    }
    return items[0]
}

// Usage - type inferred
identity(42)            // T = int
identity("hello")       // T = string
swap((1, "one"))        // T = int, U = string
```

11.1.2 Generic Types

```
struct Pair<T, U> {
    first: T
    second: U
}

struct Stack<T> {
    items: []T
}

```

```
fn (s: *Stack<T>) push<T>(item: T) {
    s.items = array.push(s.items, item)
}

fn (s: *Stack<T>) pop<T>() -> T? {
    if array.len(s.items) == 0 {
        return nil
    }
    s.items, item = array.pop(s.items)
    return item
}

// Usage
stack: Stack<int> = Stack{items: []}
stack.push(1)
stack.push(2)
value = stack.pop()  // 2
```

11.1.3 Type Constraints

Constrain generic types to those implementing certain behaviors:

```
constraint Comparable {
    fn compare(other: Self) -> int
}

constraint Printable {
    fn to_string() -> string
}

constraint Hashable {
    fn hash() -> int
}

// Constrained generic
fn max<T: Comparable>(a: T, b: T) -> T {
    if a.compare(b) > 0 {
        return a
    }
    return b
}

// Multiple constraints
fn process<T: Comparable + Printable>(items: []T) {
    sorted = array.sort(items)
    for item in sorted {
        io.println(item.to_string())
    }
}
```

11.1.4 Where Clauses

For complex constraints:

```
fn merge<K, V>(a: [K:V], b: [K:V]) -> [K:V]
```

```
    where K: Hashable + Comparable {
    result = a
    for k, v in b {
        result[k] = v
    }
    return result
}
```

11.2 Constraints (Traits)

Constraints define shared behavior that types can implement.

11.2.1 Defining Constraints

```
constraint Iterator<T> {
    fn next() -> T?
    fn has_next() -> bool
}

constraint Serializable {
    fn serialize() -> []byte
    fn deserialize(data: []byte) -> (Self, Error?)
}

constraint Ordered {
    fn less_than(other: Self) -> bool
    fn greater_than(other: Self) -> bool
    fn equals(other: Self) -> bool
}
```

11.2.2 Implementing Constraints

```
struct Counter {
    current: int
    max: int
}

impl Iterator<int> for Counter {
    fn next() -> int? {
        if self.current >= self.max {
            return nil
        }
        value = self.current
        self.current += 1
        return value
    }

    fn has_next() -> bool {
        return self.current < self.max
    }
}

// Usage
```

```
counter = Counter{current: 0, max: 5}
while counter.has_next() {
    io.println(counter.next())
}
```

11.2.3 Built-in Constraints

```
constraint Eq {
    fn equals(other: Self) -> bool
}

constraint Ord: Eq {
    fn compare(other: Self) -> int  // -1, 0, or 1
}

constraint Hash {
    fn hash() -> int
}

constraint Clone {
    fn clone() -> Self
}

constraint Default {
    fn default() -> Self
}

constraint Display {
    fn display() -> string
}

constraint Debug {
    fn debug() -> string
}
```

11.3 Union Types

Union types represent values that can be one of several types.

11.3.1 Simple Unions

```
type StringOrInt = string | int

fn process(value: StringOrInt) {
    match value {
        s: string => io.println("String: " + s)
        n: int => io.println("Int: " + to_string(n))
    }
}

process("hello")
process(42)
```

11.3.2 Tagged Unions (Enums with Data)

```
enum Result<T, E> {
    Ok(T),
    Err(E)
}

enum Option<T> {
    Some(T),
    None
}

enum Json {
    Null,
    Bool(bool),
    Number(float),
    String(string),
    Array([]Json),
    Object([string:Json])
}

// Pattern matching
fn process_json(json: Json) {
    match json {
        Json.Null => io.println("null")
        Json.Bool(b) => io.println("bool: " + to_string(b))
        Json.Number(n) => io.println("number: " + to_string(n))
        Json.String(s) => io.println("string: " + s)
        Json.Array(items) => {
            for item in items {
                process_json(item)
            }
        }
        Json.Object(obj) => {
            for key, value in obj {
                io.print(key + ": ")
                process_json(value)
            }
        }
    }
}
```

11.4 Pattern Matching

Advanced pattern matching features.

11.4.1 Destructuring

```
// Tuple destructuring
(x, y, z) = get_coordinates()

// Struct destructuring
User{name, email, age} = get_user()

```

```
// Nested destructuring
Point{x: Point{x: inner_x, y: _}, y: _} = nested_point

// Array destructuring
[first, second, ...rest] = items
[head, ...tail] = list
```

11.4.2 Pattern Guards

```
match value {
    n if n < 0 => io.println("negative")
    n if n == 0 => io.println("zero")
    n if n > 0 and n < 100 => io.println("small positive")
    n if n >= 100 => io.println("large")
}
```

11.4.3 Or Patterns

```
match status_code {
    200 | 201 | 204 => io.println("success")
    400 | 404 | 422 => io.println("client error")
    500 | 502 | 503 => io.println("server error")
    _ => io.println("unknown")
}
```

11.4.4 Binding Patterns

```
match user {
    User{name: n, age: a} if a >= 18 => {
        io.println(n + " is an adult")
    }
    User{name: n, age: a} => {
        io.println(n + " is " + to_string(a) + " years old")
    }
}
```

11.5 Operator Overloading

Types can implement operators through constraints.

```
constraint Add<T> {
    fn add(other: T) -> T
}

constraint Sub<T> {
    fn sub(other: T) -> T
}

constraint Mul<T> {
```

```
    fn mul(other: T) -> T
}

constraint Div<T> {
    fn div(other: T) -> T
}

struct Vector2 {
    x: float
    y: float
}

impl Add<Vector2> for Vector2 {
    fn add(other: Vector2) -> Vector2 {
        return Vector2{
            x: self.x + other.x,
            y: self.y + other.y
        }
    }
}

// Usage: a + b calls a.add(b)
v1 = Vector2{x: 1.0, y: 2.0}
v2 = Vector2{x: 3.0, y: 4.0}
v3 = v1 + v2  // Vector2{x: 4.0, y: 6.0}
```

11.6 Compile-Time Evaluation

Constant expressions evaluated at compile time.

```
const PI = 3.14159265358979
const TAU = PI * 2.0
const MAX_BUFFER_SIZE = 1024 * 1024

// Compile-time functions
const fn factorial(n: int) -> int {
    if n <= 1 {
        return 1
    }
    return n * factorial(n - 1)
}

const FACTORIAL_10 = factorial(10)  // Computed at compile time
```

11.7 Attributes

Attributes modify declaration behavior.

```
// Deprecation
#[deprecated("Use new_function instead")]
fn old_function() { }

// Conditional compilation
#[cfg(os = "linux")]
```

```
fn linux_specific() { }

#[cfg(debug)]
fn debug_only() { }

// Inlining hints
#[inline]
fn small_function() { }

#[inline(never)]
fn large_function() { }

// Testing
#[test]
fn test_something() { }

// Serialization
#[derive(Serialize, Deserialize)]
struct Config {
    #[rename("server_port")]
    port: int

    #[skip]
    internal_state: InternalState
}
```

11.8 Macros

Compile-time code generation.

```
// Simple macro
macro debug!(expr) {
    io.println(stringify!(expr) + " = " + to_string(expr))
}

debug!(x + y)  // Expands to: io.println("x + y" + " = " +
    to_string(x + y))

// Variadic macro
macro println!(format, ...args) {
    io.printf(format + "\n", args...)
}

println!("Hello, %s!", name)

// Assert macro
macro assert!(condition, message = "assertion failed") {
    if not condition {
        panic(message + " at " + file!() + ":" + line!())
    }
}
```

11.9 Unsafe Code

For low-level operations.

```
unsafe {
    // Raw pointer operations
    ptr = raw_pointer(data)
    value = *ptr
    *ptr = new_value

    // FFI calls
    result = c_function(arg1, arg2)
}

// Unsafe function
unsafe fn direct_memory_access(addr: usize) -> u8 {
    ptr = addr as *u8
    return *ptr
}
```

Warning

Unsafe code bypasses HAIRA's safety guarantees. Use only when necessary and document thoroughly.

11.10 Foreign Function Interface (FFI)

Calling C code.

```
// Declare external function
extern "C" {
    fn printf(format: *i8, ...) -> i32
    fn malloc(size: usize) -> *void
    fn free(ptr: *void)
}

// Use in unsafe block
unsafe {
    ptr = malloc(1024)
    // Use memory...
    free(ptr)
}

// Wrapper for safe interface
fn safe_alloc(size: int) -> []byte {
    unsafe {
        ptr = malloc(size as usize)
        return bytes_from_raw(ptr, size)
    }
}
```

11.11 Reflection

Runtime type information.

```
import "reflect"

fn describe(value: any) {
    t = reflect.type_of(value)
    io.println("Type: " + t.name)
    io.println("Kind: " + t.kind)

    if t.kind == "struct" {
        for field in t.fields {
            io.println("  " + field.name + ": " +
                field.type.name)
        }
    }
}

struct User {
    name: string
    age: int
}

describe(User{name: "Alice", age: 30})
// Output:
// Type: User
// Kind: struct
//   name: string
//   age: int
```

11.12 Advanced Features Grammar

```
generic_params = "<" type_param { "," type_param } ">" ;

type_param = identifier [ ":" constraint_list ] ;

constraint_list = identifier { "+" identifier } ;

where_clause = "where" where_item { "," where_item } ;

where_item = identifier ":" constraint_list ;

constraint_def = "constraint" identifier [ generic_params ] "{"
                 { fn_signature }
                 "}" ;

impl_block = "impl" [ generic_params ] identifier "for" type
             [ where_clause ] "{" { fn_def } "}" ;

union_type = type { "|" type } ;

attribute = "#[" identifier [ "(" attr_args ")" ] "]" ;
```

Part III

AI Integration

Chapter 12

AI Integration

HAIRA provides first-class AI integration for code generation. AI blocks allow developers to describe intent in natural language, with implementations generated at compile time.

12.1 Philosophy

- **Explicit invocation** – AI is only used when explicitly requested via `ai` blocks
- **Compile-time generation** – All AI processing happens during compilation
- **Local-first** – Prefer local models (llama.cpp, Ollama) over cloud APIs
- **Deterministic builds** – Generated code is cached for reproducibility
- **Type-safe output** – Generated code must match declared signatures

12.2 AI Block Syntax

```
ai function_name(param1: Type1, param2: Type2) -> ReturnType {
    Natural language description of what the function should do.
    Can span multiple lines.
    Include examples and constraints as needed.
}
```

12.2.1 Basic Example

```
import "io"

struct User {
    name: string
    email: string
    created_at: int
}

ai format_user(user: User) -> string {
    Format the user information into a human-readable string.
    Include the name, email, and how long ago they registered.
    Example: "Alice (alice@example.com) - joined 3 days ago"
}
```

```

fn main() {
    user = User{
        name: "Alice",
        email: "alice@example.com",
        created_at: 1709251200
    }
    io.println(format_user(user))
}
```

12.2.2 Complex Example

```
struct Transaction {
    id: string
    amount: float
    currency: string
    timestamp: int
    category: string
}

ai analyze_spending(transactions: []Transaction) ->
   SpendingReport {
    Analyze the list of transactions and generate a spending
       report.

    The report should include:
    - Total spending per category
    - Average transaction amount
    - Highest and lowest transactions
    - Month-over-month comparison if data spans multiple months

    Handle edge cases:
    - Empty transaction list should return a report with zero
       values
    - Mixed currencies should be converted to USD using
       approximate rates
}

struct SpendingReport {
    by_category: [string:float]
    average_amount: float
    highest: Transaction?
    lowest: Transaction?
    mom_change: float?
}
```

12.3 AI Backend Configuration

Configure AI backends in `haira.toml`:

```
[ai]
# Primary backend (choose one)
backend = "llama.cpp"    # Local llama.cpp
# backend = "ollama"     # Local Ollama server
```

```
# backend = "anthropic"  # Anthropic API
# backend = "openai"     # OpenAI API

# llama.cpp settings
[ai.llama]
model_path = "~/.haira/models/codellama-7b.gguf"
context_size = 4096
threads = 8
gpu_layers = 32

# Ollama settings
[ai.ollama]
host = "localhost:11434"
model = "deepseek-coder-v2:16b"
timeout = 60

# Anthropic settings
[ai.anthropic]
api_key = "${ANTHROPIC_API_KEY}"
model = "claude-sonnet-4-20250514"

# OpenAI settings
[ai.openai]
api_key = "${OPENAI_API_KEY}"
model = "gpt-4"

# Caching
[ai.cache]
enabled = true
directory = ".haira/ai-cache"
```

12.3.1 Backend Priority

When multiple backends are available:

1. Try configured primary backend
2. Fall back to other local backends
3. Fall back to cloud backends (if configured)
4. Error if no backend available

12.3.2 CLI Overrides

```
# Use specific backend
haira build main.haira --ai-backend=ollama

# Offline mode (cache only)
haira build main.haira --offline

# Regenerate all AI blocks
haira build main.haira --refresh-ai

# Disable AI (error on ai blocks)
haira build main.haira --no-ai
```

12.4 Compile-Time Processing

12.4.1 Generation Flow

1. Parser encounters `ai` block
2. Extract: function signature, intent description, surrounding context
3. Generate cache key: `sha256(intent + signature + context)`
4. Check cache for existing generation
5. If not cached: invoke AI backend with structured prompt
6. Parse generated code (CIR format)
7. Validate against declared signature
8. Insert into AST as regular function
9. Cache result for future builds

12.4.2 Context Awareness

AI blocks have access to:

- Function signature (parameters, return type)
- Type definitions used in signature
- Imported modules and their public interfaces
- Other functions in the same file

```
struct User {
    name: string
    posts: []Post
}

struct Post {
    title: string
    likes: int
}

// AI can see User and Post definitions
ai get_popular_posts(user: User, min_likes: int) -> []Post {
    Return all posts from the user that have at least min_likes
       likes.
    Sort by likes in descending order.
}
```

12.5 CIR: Canonical Intermediate Representation

AI backends output CIR, a JSON-based intermediate format that represents Haira code.

12.5.1 CIR Structure

```
{
  "version": "1.0",
  "body": [
    {
      "type": "statement",
      "kind": "variable_decl",
      "name": "result",
      "value": { ... }
    },
    {
      "type": "statement",
      "kind": "return",
      "value": { ... }
    }
  ]
}
```

See Chapter 13 for the complete CIR specification.

12.5.2 Why CIR?

- **Structured output** – Easier for AI to generate correctly
- **Validation** – Can validate structure before code generation
- **Language evolution** – CIR can remain stable as syntax changes
- **Debugging** – Easier to inspect and debug than raw code

12.6 Caching and Reproducibility

12.6.1 Cache Key Generation

```
cache_key = sha256(
    intent_description +
    function_signature +
    type_definitions +
    haira_version +
    ai_backend_version
)
```

12.6.2 Cache Location

```
.haira/
  ai-cache/
    ab12cd34.json      # Cached CIR
    ab12cd34.meta      # Metadata (timestamps, backend used)
```

12.6.3 Cache Commands

```
# View cache statistics
haira ai cache-stats

# Clear cache
haira ai cache-clear

# Export cache (for CI/CD)
haira ai cache-export --output cache.tar.gz

# Import cache
haira ai cache-import cache.tar.gz
```

12.7 Best Practices

12.7.1 When to Use AI Blocks

Good use cases:

- Data transformation and formatting
- Text processing and parsing
- Business logic with clear rules
- Utility functions with well-defined behavior

Poor use cases:

- Security-critical code
- Performance-critical hot paths
- Code requiring external state or I/O
- Complex algorithms with specific requirements

12.7.2 Writing Good Intent Descriptions

```
1  // BAD: Too vague
2  ai process(data: []int) -> int {
3      Process the data.
4  }
5
6  // GOOD: Specific and detailed
7  ai calculate_median(numbers: []int) -> float {
8      Calculate the median value of the given numbers.
9
10     For odd-length arrays, return the middle element.
11     For even-length arrays, return the average of the two
           middle elements.
12
13     Handle edge cases:
14     - Empty array: return 0.0
15     - Single element: return that element as float
```

```
16
17      Example:
18      - [1, 3, 5] -> 3.0
19      - [1, 2, 3, 4] -> 2.5
20  }
```

12.7.3 Testing AI-Generated Code

```
1   ai parse_date(s: string) -> (Date, Error?) {
2       Parse a date string in various formats:
3       - "2024-03-15"
4       - "March 15, 2024"
5       - "15/03/2024"
6       Return error for invalid formats.
7   }
8
9   #[test]
10  fn test_parse_date() {
11      // Test ISO format
12      date, err = parse_date("2024-03-15")
13      assert(err == nil)
14      assert(date.year == 2024)
15      assert(date.month == 3)
16      assert(date.day == 15)
17
18      // Test invalid format
19      _, err = parse_date("not a date")
20      assert(err != nil)
21  }
```

12.8 Error Handling

12.8.1 Compilation Errors

```
1   error[AI001]: AI generation failed
2     --> src/main.haira:15:1
3      |
4   15 | ai complex_function(...) -> Result {
5      | ^^^ AI backend returned invalid CIR
6      |
7      = note: Backend error: "Unable to generate implementation"
8      = help: Try simplifying the intent description
9
10  error[AI002]: Generated code type mismatch
11    --> src/main.haira:20:1
12     |
13  20 | ai transform(x: int) -> string {
14     | ^^^ Expected return type `string`, got `int`
15     |
16     = note: AI generated code returning wrong type
17     = help: Regenerate with --refresh-ai
```

12.8.2 Fallback Behavior

```
// Provide fallback implementation
ai smart_parse(input: string) -> Data {
    Parse the input intelligently, handling various formats.
} fallback {
    // Manual implementation if AI fails
    return basic_parse(input)
}
```

12.9 AI Block Grammar

```
ai_block = "ai" identifier "(" [ params ] ")" [ "->" type ]
           "{" intent_description "}"
           [ "fallback" block ] ;

intent_description = { any_char } ;  (* Free-form text *)
```

Chapter 13

CIR Specification

The Canonical Intermediate Representation (CIR) is a JSON-based format for representing Haira code. It serves as the output format for AI code generation.

13.1 Overview

CIR provides a structured, validated representation of code that is:

- Easy for AI models to generate correctly
- Simple to validate before compilation
- Version-stable across language evolution
- Human-readable for debugging

13.2 Document Structure

```
{
  "version": "1.0",
  "body": [ <statement>, ... ]
}
```

13.2.1 Version Field

The `version` field indicates the CIR schema version. Current version is `"1.0"`.

13.2.2 Body Field

The `body` field contains an array of statements representing the function body.

13.3 Statements

13.3.1 Variable Declaration

```
{
  "type": "statement",
  "kind": "variable_decl",
```

```
  "name": "x",
  "type_annotation": "int",  // optional
  "value": <expression>
}
```

13.3.2 Assignment

```
{
  "type": "statement",
  "kind": "assignment",
  "target": <lvalue>,
  "operator": "=",  // or "+=", "-=", "*=", "/="
  "value": <expression>
}
```

13.3.3 Return

```
{
  "type": "statement",
  "kind": "return",
  "value": <expression>  // optional for void functions
}
```

13.3.4 If Statement

```
{
  "type": "statement",
  "kind": "if",
  "condition": <expression>,
  "then_body": [ <statement>, ... ],
  "else_body": [ <statement>, ... ]  // optional
}
```

13.3.5 For Loop

```
{
  "type": "statement",
  "kind": "for",
  "variable": "i",
  "index_variable": "idx",  // optional, for indexed iteration
  "iterable": <expression>,
  "body": [ <statement>, ... ]
}
```

13.3.6 While Loop

```
{
  "type": "statement",
  "kind": "while",
  "condition": <expression>,
  "body": [ <statement>, ... ]
}
```

13.3.7 Match Statement

```
{
  "type": "statement",
  "kind": "match",
  "value": <expression>,
  "arms": [
    {
      "pattern": <pattern>,
      "guard": <expression>,  // optional
      "body": [ <statement>, ... ]
    },
    ...
  ]
}
```

13.3.8 Expression Statement

```
{
  "type": "statement",
  "kind": "expression",
  "expression": <expression>
}
```

13.3.9 Break

```
{
  "type": "statement",
  "kind": "break",
  "label": "outer"  // optional
}
```

13.3.10 Continue

```
{
  "type": "statement",
  "kind": "continue",
  "label": "outer"  // optional
}
```

13.4 Expressions

13.4.1 Literals

```
// Integer
{
  "type": "expression",
  "kind": "literal",
  "literal_type": "int",
  "value": 42
}

// Float
{
  "type": "expression",
  "kind": "literal",
  "literal_type": "float",
  "value": 3.14
}

// String
{
  "type": "expression",
  "kind": "literal",
  "literal_type": "string",
  "value": "hello"
}

// Boolean
{
  "type": "expression",
  "kind": "literal",
  "literal_type": "bool",
  "value": true
}

// Nil
{
  "type": "expression",
  "kind": "literal",
  "literal_type": "nil"
}
```

13.4.2 Identifier

```
{
  "type": "expression",
  "kind": "identifier",
  "name": "variable_name"
}
```

13.4.3 Binary Operation

```
{
  "type": "expression",
  "kind": "binary_op",
  "operator": "+",  // +, -, *, /, %, ==, !=, <, >, <=, >=,
     and, or
  "left": <expression>,
  "right": <expression>
}
```

13.4.4 Unary Operation

```
{
  "type": "expression",
  "kind": "unary_op",
  "operator": "-",  // -, !, not
  "operand": <expression>
}
```

13.4.5 Function Call

```
{
  "type": "expression",
  "kind": "call",
  "function": <expression>,  // identifier or field access
  "arguments": [ <expression>, ... ]
}
```

13.4.6 Method Call

```
{
  "type": "expression",
  "kind": "method_call",
  "receiver": <expression>,
  "method": "method_name",
  "arguments": [ <expression>, ... ]
}
```

13.4.7 Field Access

```
{
  "type": "expression",
  "kind": "field_access",
  "object": <expression>,
  "field": "field_name"
}
```

13.4.8 Index Access

```
{
  "type": "expression",
  "kind": "index_access",
  "object": <expression>,
  "index": <expression>
}
```

13.4.9 Array Literal

```
{
  "type": "expression",
  "kind": "array",
  "elements": [ <expression>, ... ]
}
```

13.4.10 Map Literal

```
{
  "type": "expression",
  "kind": "map",
  "entries": [
    { "key": <expression>, "value": <expression> },
    ...
  ]
}
```

13.4.11 Struct Literal

```
{
  "type": "expression",
  "kind": "struct",
  "type_name": "User",
  "fields": [
    { "name": "id", "value": <expression> },
    { "name": "name", "value": <expression> }
  ]
}
```

13.4.12 If Expression

```
{
  "type": "expression",
  "kind": "if_expr",
  "condition": <expression>,
  "then_value": <expression>,
  "else_value": <expression>
```

```
}
```

13.4.13 Match Expression

```
{
  "type": "expression",
  "kind": "match_expr",
  "value": <expression>,
  "arms": [
    {
      "pattern": <pattern>,
      "guard": <expression>,  // optional
      "value": <expression>
    },
    ...
  ]
}
```

13.4.14 Range

```
{
  "type": "expression",
  "kind": "range",
  "start": <expression>,
  "end": <expression>,
  "inclusive": false  // true for ..=
}
```

13.4.15 Closure

```
{
  "type": "expression",
  "kind": "closure",
  "parameters": [
    { "name": "x", "type": "int" }
  ],
  "return_type": "int",  // optional
  "body": [ <statement>, ... ]
}
```

13.5 Patterns

13.5.1 Literal Pattern

```
{
  "type": "pattern",
  "kind": "literal",
  "value": 42
```

```
}
```

13.5.2 Identifier Pattern

```
{
  "type": "pattern",
  "kind": "identifier",
  "name": "x"
}
```

13.5.3 Wildcard Pattern

```
{
  "type": "pattern",
  "kind": "wildcard"
}
```

13.5.4 Struct Pattern

```
{
  "type": "pattern",
  "kind": "struct",
  "type_name": "User",
  "fields": [
    { "name": "id", "pattern": <pattern> },
    { "name": "name", "pattern": <pattern> }
  ]
}
```

13.5.5 Tuple Pattern

```
{
  "type": "pattern",
  "kind": "tuple",
  "elements": [ <pattern>, ... ]
}
```

13.5.6 Or Pattern

```
{
  "type": "pattern",
  "kind": "or",
  "patterns": [ <pattern>, ... ]
}
```

13.5.7 Range Pattern

```
{
  "type": "pattern",
  "kind": "range",
  "start": 0,
  "end": 10,
  "inclusive": true
}
```

13.6 L-Values

For assignment targets:

```
// Identifier
{
  "type": "lvalue",
  "kind": "identifier",
  "name": "x"
}

// Field access
{
  "type": "lvalue",
  "kind": "field",
  "object": <expression>,
  "field": "name"
}

// Index access
{
  "type": "lvalue",
  "kind": "index",
  "object": <expression>,
  "index": <expression>
}
```

13.7 Complete Example

Haira code:

```
ai sum_positive(numbers: []int) -> int {
    Return the sum of all positive numbers in the array.
}
```

Generated CIR:

```
{
  "version": "1.0",
  "body": [
    {
      "type": "statement",
      "kind": "variable_decl",
```

```
      "name": "total",
      "value": {
        "type": "expression",
        "kind": "literal",
        "literal_type": "int",
        "value": 0
      }
    },
    {
      "type": "statement",
      "kind": "for",
      "variable": "n",
      "iterable": {
        "type": "expression",
        "kind": "identifier",
        "name": "numbers"
      },
      "body": [
        {
          "type": "statement",
          "kind": "if",
          "condition": {
            "type": "expression",
            "kind": "binary_op",
            "operator": ">",
            "left": {
              "type": "expression",
              "kind": "identifier",
              "name": "n"
            },
            "right": {
              "type": "expression",
              "kind": "literal",
              "literal_type": "int",
              "value": 0
            }
          },
          "then_body": [
            {
              "type": "statement",
              "kind": "assignment",
              "target": {
                "type": "lvalue",
                "kind": "identifier",
                "name": "total"
              },
              "operator": "+=",
              "value": {
                "type": "expression",
                "kind": "identifier",
                "name": "n"
              }
            }
          ]
        }
      ]
    },
    {
```

```
      "type": "statement",
      "kind": "return",
      "value": {
        "type": "expression",
        "kind": "identifier",
        "name": "total"
      }
    }
  ]
}
```

13.8 Validation Rules

CIR must satisfy these rules:

1. All referenced identifiers must be in scope (parameters or previously declared)
2. Return statements must match declared return type
3. Binary operators must be valid for operand types
4. Field access must reference valid struct fields
5. Index access must be on array or map types
6. Match expressions must be exhaustive

13.9 JSON Schema

The complete JSON Schema for CIR validation is available at:

```
https://haira-lang.org/schemas/cir-1.0.json
```

13.10 Error Codes

Code	Description
CIR001	Invalid version
CIR002	Missing required field
CIR003	Unknown node kind
CIR004	Type mismatch
CIR005	Undefined identifier
CIR006	Invalid operator
CIR007	Non-exhaustive match

Table 13.1: CIR validation error codes

Part IV

Implementation

Chapter 14

Complete Grammar

This chapter provides the complete formal grammar for Haira in Extended Backus-Naur Form (EBNF).

14.1 Notation

Notation	Meaning
`=`	Definition
`|`	Alternative
`[ ... ]`	Optional (0 or 1)
`{ ... }`	Repetition (0 or more)
`( ... )`	Grouping
`"..."`	Terminal string
`;`	End of production

Table 14.1: EBNF notation

14.2 Lexical Grammar

14.2.1 Characters

```
letter        = "a".."z" | "A".."Z" | "_" ;
digit         = "0".."9" ;
hex_digit     = digit | "a".."f" | "A".."F" ;
octal_digit   = "0".."7" ;
binary_digit  = "0" | "1" ;
```

14.2.2 Whitespace and Comments

```
whitespace    = " " | "\t" | "\r" ;
newline       = "\n" ;
line_comment  = "//" { any_char } newline ;
block_comment = "/*" { any_char | block_comment } "*/" ;
doc_comment   = "///" { any_char } newline ;
```

14.2.3 Identifiers

```
identifier     = letter { letter | digit } ;
```

14.2.4 Keywords

```
keyword        = "ai" | "and" | "as" | "break" | "catch"
               | "chan" | "const" | "continue" | "defer"
               | "else" | "enum" | "false" | "fn" | "for"
               | "from" | "if" | "impl" | "import" | "in"
               | "match" | "nil" | "not" | "or" | "return"
               | "select" | "spawn" | "struct" | "true"
               | "try" | "type" | "unsafe" | "where" | "while" ;
```

14.2.5 Literals

```
integer_lit    = decimal_lit | hex_lit | octal_lit | binary_lit ;
decimal_lit    = digit { digit | "_" } ;
hex_lit        = "0" ("x"|"X") hex_digit { hex_digit | "_" } ;
octal_lit      = "0" ("o"|"O") octal_digit { octal_digit | "_" }
    ;
binary_lit     = "0" ("b"|"B") binary_digit { binary_digit | "_"
    } ;

float_lit      = decimal_lit "." decimal_lit [ exponent ]
               | decimal_lit exponent ;
exponent       = ("e"|"E") ["+"|"-"] decimal_lit ;

string_lit     = '"' { string_char } '"'
               | "r" '"' { raw_char } '"'
               | '"""' { any_char } '"""' ;
string_char    = any_char - ('"' | "\" | newline)
               | escape_seq ;
escape_seq     = "\" ( "n" | "r" | "t" | "\" | '"' | "0"
               | "x" hex_digit hex_digit
               | "u" "{" hex_digit { hex_digit } "}" ) ;
raw_char       = any_char - '"' ;

bool_lit       = "true" | "false" ;
nil_lit        = "nil" ;
```

14.2.6 Operators

```
operator       = arith_op | comp_op | logic_op | assign_op |
    other_op ;

arith_op       = "+" | "-" | "*" | "/" | "%" ;
comp_op        = "==" | "!=" | "<" | ">" | "<=" | ">=" ;
logic_op       = "&&" | "||" | "!" ;
assign_op      = "=" | "+=" | "-=" | "*=" | "/=" ;
```

```
other_op      = "|" | ".." | "..=" | "<-" | "->" | "=>" | "?" ;
```

14.2.7 Delimiters

```
delimiter     = "(" | ")" | "[" | "]" | "{" | "}"
              | "," | ":" | "." | ";" ;
```

14.3 Syntactic Grammar

14.3.1 Program Structure

```
program       = { import_stmt } { top_level } ;

top_level     = fn_decl
              | struct_decl
              | enum_decl
              | type_alias
              | const_decl
              | impl_block
              | constraint_decl
              | ai_block ;
```

14.3.2 Imports

```
import_stmt   = "import" import_spec ;

import_spec   = string_lit
              | identifier "from" string_lit
              | "{" ident_list "}" "from" string_lit
              | "*" "from" string_lit ;

ident_list    = identifier { "," identifier } ;
```

14.3.3 Declarations

```
fn_decl       = [ attributes ] "fn" [ receiver ] identifier
                [ generic_params ] "(" [ params ] ")"
                [ "->" type ] [ where_clause ] block ;

receiver      = "(" identifier ":" [ "*" ] type ")" ;

generic_params = "<" generic_param { "," generic_param } ">" ;

generic_param  = identifier [ ":" constraint_list ] ;

constraint_list = identifier { "+" identifier } ;

where_clause  = "where" where_bound { "," where_bound } ;
```

```
14
15 where_bound   = identifier ":" constraint_list ;
16
17 params        = param { "," param } ;
18
19 param         = identifier ":" [ "..." ] type [ "=" expression
   ] ;
20
21 struct_decl   = [ attributes ] "struct" identifier
22                 [ generic_params ] "{" { field_decl } "}" ;
23
24 field_decl    = [ attributes ] identifier ":" type ;
25
26 enum_decl     = [ attributes ] "enum" identifier
27                 [ generic_params ] "{" enum_variants "}" ;
28
29 enum_variants = enum_variant { "," enum_variant } [ "," ] ;
30
31 enum_variant  = identifier [ "(" type_list ")" ] ;
32
33 type_alias    = "type" identifier [ generic_params ] "=" type ;
34
35 const_decl    = "const" identifier [ ":" type ] "=" expression ;
36
37 impl_block    = "impl" [ generic_params ] identifier
38                 "for" type [ where_clause ]
39                 "{" { fn_decl } "}" ;
40
41 constraint_decl = "constraint" identifier [ generic_params ]
42                   "{" { fn_signature } "}" ;
43
44 fn_signature  = "fn" identifier "(" [ type_params ] ")" [ "->"
   type ] ;
45
46 type_params   = type { "," type } ;
47
48 ai_block      = "ai" identifier "(" [ params ] ")" [ "->" type ]
49                 "{" intent_text "}" [ "fallback" block ] ;
50
51 intent_text   = { any_char } ;
52
53 attributes    = { attribute } ;
54
55 attribute     = "#[" identifier [ "(" attr_args ")" ] "]" ;
56
57 attr_args     = attr_arg { "," attr_arg } ;
58
59 attr_arg      = identifier [ "=" literal ] ;
```

14.3.4 Types

```
1 type          = simple_type
2               | array_type
3               | map_type
4               | tuple_type
5               | option_type
```

```
                | fn_type
                | generic_type
                | pointer_type
                | union_type ;

simple_type     = "int" | "i8" | "i16" | "i32" | "i64"
                | "u8" | "u16" | "u32" | "u64"
                | "float" | "f32" | "f64"
                | "bool" | "string"
                | identifier ;

array_type      = "[" "]" type ;

map_type        = "[" type ":" type "]" ;

tuple_type      = "(" type { "," type } ")" ;

option_type     = type "?" ;

fn_type         = "fn" "(" [ type_list ] ")" [ "->" type ] ;

type_list       = type { "," type } ;

generic_type    = identifier "<" type_list ">" ;

pointer_type    = "*" type ;

union_type      = type { "|" type } ;

channel_type    = "chan" "<" type ">"
                | "chan<-" type
                | "<-chan<" type ">" ;
```

14.3.5 Statements

```
statement       = var_decl
                | assignment
                | expr_stmt
                | if_stmt
                | for_stmt
                | while_stmt
                | match_stmt
                | return_stmt
                | break_stmt
                | continue_stmt
                | defer_stmt
                | spawn_stmt
                | send_stmt
                | select_stmt
                | block ;

var_decl        = ident_list [ ":" type ] "=" expr_list ;

ident_list      = identifier { "," identifier } ;

expr_list       = expression { "," expression } ;
```

```
22
23 assignment    = lvalue_list assign_op expr_list ;
24
25 lvalue_list   = lvalue { "," lvalue } ;
26
27 lvalue        = identifier
28               | lvalue "." identifier
29               | lvalue "[" expression "]" ;
30
31 expr_stmt     = expression ;
32
33 if_stmt       = "if" [ simple_stmt ";" ] expression block
34                 [ "else" ( if_stmt | block ) ] ;
35
36 simple_stmt   = var_decl | assignment | expr_stmt ;
37
38 for_stmt      = "for" [ identifier "," ] identifier "in"
   expression block ;
39
40 while_stmt    = "while" expression block ;
41
42 match_stmt    = "match" expression "{" { match_arm } "}" ;
43
44 match_arm     = pattern [ "if" expression ] "=>" ( expression |
   block ) ;
45
46 return_stmt   = "return" [ expr_list ] ;
47
48 break_stmt    = "break" [ identifier ] ;
49
50 continue_stmt = "continue" [ identifier ] ;
51
52 defer_stmt    = "defer" statement ;
53
54 spawn_stmt    = "spawn" ( block | fn_call ) ;
55
56 send_stmt     = expression "<-" expression ;
57
58 select_stmt   = "select" "{" { select_case } [ default_case ]
   "}" ;
59
60 select_case   = pattern "=" "<-" expression "=>" ( expression |
   block )
61               | expression "<-" expression "=>" ( expression |
                    block ) ;
62
63 default_case  = "default" "=>" ( expression | block ) ;
64
65 block         = "{" { statement } "}" ;
```

14.3.6 Expressions

```
1 expression    = assign_expr ;
2
3 assign_expr   = or_expr [ assign_op assign_expr ] ;
4
```

```
or_expr        = and_expr { ("or" | "||") and_expr } ;

and_expr       = equality { ("and" | "&&") equality } ;

equality       = comparison { ("==" | "!=") comparison } ;

comparison     = pipe { ("<" | ">" | "<=" | ">=") pipe } ;

pipe           = range { "|" range } ;

range          = additive [ (".." | "..=") additive ] ;

additive       = multiplicative { ("+" | "-") multiplicative } ;

multiplicative = unary { ("*" | "/" | "%") unary } ;

unary          = ("!" | "-" | "not") unary
               | postfix ;

postfix        = primary { postfix_op } ;

postfix_op     = call
               | index
               | field
               | optional_chain ;

call           = "(" [ arguments ] ")" ;

arguments      = argument { "," argument } ;

argument       = [ identifier ":" ] expression ;

index          = "[" expression "]" ;

field          = "." identifier ;

optional_chain = "?" "." identifier ;

primary        = identifier
               | literal
               | "(" expression ")"
               | array_lit
               | map_lit
               | struct_lit
               | closure
               | if_expr
               | match_expr
               | receive_expr
               | block ;

literal        = integer_lit | float_lit | string_lit
               | bool_lit | nil_lit ;

array_lit      = "[" [ expr_list ] "]" ;

map_lit        = "[" ":" "]"
               | "[" map_entry { "," map_entry } [ "," ] "]" ;

```

```
map_entry     = expression ":" expression ;

struct_lit    = identifier "{" [ field_inits ] "}" ;

field_inits   = field_init { "," field_init } [ "," ] ;

field_init    = identifier [ ":" expression ] ;

closure       = "fn" "(" [ closure_params ] ")" [ "->" type ]
                ( block | "{" expression "}" ) ;

closure_params = closure_param { "," closure_param } ;

closure_param  = identifier [ ":" type ] ;

if_expr       = "if" expression block "else" ( if_expr | block ) ;

match_expr    = "match" expression "{" { match_arm } "}" ;

receive_expr  = "<-" expression ;
```

14.3.7 Patterns

```
pattern       = literal_pat
              | ident_pat
              | wildcard_pat
              | tuple_pat
              | struct_pat
              | enum_pat
              | array_pat
              | range_pat
              | or_pat ;

literal_pat   = integer_lit | float_lit | string_lit | bool_lit ;

ident_pat     = identifier ;

wildcard_pat  = "_" ;

tuple_pat     = "(" pattern { "," pattern } ")" ;

struct_pat    = identifier "{" [ field_pats ] "}" ;

field_pats    = field_pat { "," field_pat } ;

field_pat     = identifier [ ":" pattern ] ;

enum_pat      = identifier "." identifier [ "(" [ pattern_list ] ")" ] ;

pattern_list  = pattern { "," pattern } ;

array_pat     = "[" [ pattern_list ] [ "," "..." [ identifier ] ] "]" ;
```

```
30
31 range_pat      = literal ".." [ "=" ] literal ;
32
33 or_pat         = pattern { "|" pattern } ;
```

14.4 Grammar Summary

14.4.1 Entry Points

- `program` – Complete source file
- `expression` – Single expression (REPL)
- `statement` – Single statement
- `type` – Type annotation

14.4.2 Precedence (High to Low)

1. Postfix: `() [] . ?.`
2. Unary: `- ! not`
3. Multiplicative: `* / %`
4. Additive: `+ -`
5. Range: `.. ..=`
6. Pipe: `|`
7. Comparison: `< > <= >=`
8. Equality: `== !=`
9. Logical AND: `and &&`
10. Logical OR: `or ||`
11. Assignment: `= += -= *= /=`

14.4.3 Associativity

- Left-associative: All binary operators except assignment
- Right-associative: Assignment operators, unary operators
- Non-associative: Comparison operators, range operators

Chapter 15

Compiler Architecture

This chapter describes the architecture and implementation of the Haira compiler.

15.1 Overview

The Haira compiler (`hairac`) transforms source code into native executables. It is written in Rust and uses Cranelift for code generation.

```
Source (.haira)
       |
       v
   +---------+
   | Lexer   |  --> Tokens
   +---------+
       |
       v
   +---------+
   | Parser  |  --> AST
   +---------+
       |
       v
   +-----------+
   | Resolver  |  --> Resolved AST (imports, symbols)
   +-----------+
       |
       v
   +--------------+
   | Type Checker |  --> Typed AST
   +--------------+
       |
       v
   +-----------+
   | AI Phase  |  --> AI blocks expanded (optional)
   +-----------+
       |
       v
   +-----------+
   | Lowering  |  --> HIR (High-level IR)
   +-----------+
       |
       v
   +-----------+
   | Codegen   |  --> Cranelift IR --> Native binary
```

```
   +----------+
```

15.2 Lexer

The lexer (scanner) converts source text into a stream of tokens.

15.2.1 Token Types

```
enum TokenKind {
    // Literals
    IntLit(i64),
    FloatLit(f64),
    StringLit(String),
    BoolLit(bool),

    // Identifiers and keywords
    Ident(String),
    Keyword(Keyword),

    // Operators
    Plus, Minus, Star, Slash, Percent,
    Eq, EqEq, NotEq, Lt, Gt, LtEq, GtEq,
    And, Or, Not, AndAnd, OrOr, Bang,
    PlusEq, MinusEq, StarEq, SlashEq,
    Pipe, DotDot, DotDotEq, Arrow, FatArrow,
    LeftArrow, Question,

    // Delimiters
    LParen, RParen, LBracket, RBracket, LBrace, RBrace,
    Comma, Colon, Dot, Semicolon,

    // Special
    Newline, Eof, Error(String),
}
```

15.2.2 Lexer Implementation

```
struct Lexer<'a> {
    source: &'a str,
    chars: Peekable<CharIndices<'a>>,
    line: usize,
    column: usize,
}

impl<'a> Lexer<'a> {
    fn next_token(&mut self) -> Token {
        self.skip_whitespace();

        match self.peek_char() {
            None => Token::new(TokenKind::Eof, self.span()),
            Some(c) => match c {
                '0'..='9' => self.scan_number(),
                '"' => self.scan_string(),
```

```
                'a'..='z' | 'A'..='Z' | '_' =>
                    self.scan_identifier(),
                '+' => self.scan_operator(TokenKind::Plus,
                    TokenKind::PlusEq),
                // ... other cases
            }
        }
    }
}
```

15.3 Parser

The parser constructs an Abstract Syntax Tree (AST) from the token stream.

15.3.1 AST Nodes

```
enum Expr {
    Literal(Literal),
    Ident(Ident),
    Binary(Box<Expr>, BinOp, Box<Expr>),
    Unary(UnaryOp, Box<Expr>),
    Call(Box<Expr>, Vec<Expr>),
    FieldAccess(Box<Expr>, Ident),
    Index(Box<Expr>, Box<Expr>),
    If(Box<Expr>, Block, Option<Block>),
    Match(Box<Expr>, Vec<MatchArm>),
    Closure(Vec<Param>, Option<Type>, Block),
    Array(Vec<Expr>),
    Map(Vec<(Expr, Expr)>),
    Struct(Ident, Vec<(Ident, Expr)>),
    Range(Box<Expr>, Box<Expr>, bool),
    Pipe(Box<Expr>, Box<Expr>),
}

enum Stmt {
    VarDecl(Vec<Ident>, Option<Type>, Vec<Expr>),
    Assignment(Vec<LValue>, AssignOp, Vec<Expr>),
    Expr(Expr),
    If(Expr, Block, Option<Block>),
    For(Option<Ident>, Ident, Expr, Block),
    While(Expr, Block),
    Match(Expr, Vec<MatchArm>),
    Return(Option<Vec<Expr>>),
    Break(Option<Ident>),
    Continue(Option<Ident>),
    Defer(Box<Stmt>),
    Spawn(SpawnTarget),
    Block(Block),
}
```

15.3.2 Recursive Descent

The parser uses recursive descent with Pratt parsing for expressions:

```
impl Parser<'_> {
    fn parse_expression(&mut self) -> Result<Expr, ParseError> {
        self.parse_expr_bp(0)
    }

    fn parse_expr_bp(&mut self, min_bp: u8) -> Result<Expr,
       ParseError> {
        let mut lhs = self.parse_prefix()?;

        loop {
            let op = match self.peek() {
                Some(Token { kind: TokenKind::Plus, .. }) =>
                   BinOp::Add,
                // ... other operators
                _ => break,
            };

            let (l_bp, r_bp) = op.binding_power();
            if l_bp < min_bp {
                break;
            }

            self.advance();
            let rhs = self.parse_expr_bp(r_bp)?;
            lhs = Expr::Binary(Box::new(lhs), op,
               Box::new(rhs));
        }

        Ok(lhs)
    }
}
```

15.4 Resolver

The resolver performs:

- Import resolution
- Symbol binding (variables, functions, types)
- Scope analysis
- Forward declaration handling

15.4.1 Symbol Table

```
struct SymbolTable {
    scopes: Vec<Scope>,
    current: usize,
}

struct Scope {
    symbols: HashMap<String, Symbol>,
    parent: Option<usize>,
}
```

```

enum Symbol {
    Variable { ty: TypeId, mutable: bool },
    Function { sig: FunctionSig },
    Type { def: TypeDef },
    Module { exports: HashMap<String, Symbol> },
}
```

15.4.2 Import Resolution

```
fn resolve_import(&mut self, import: &Import) -> Result<(), ResolveError> {
    let module = match &import.path {
        // Standard library
        path if is_stdlib(path) => self.load_stdlib(path)?,

        // Project-local
        path if is_local(path) => self.load_local(path)?,

        // External package
        path => self.load_external(path)?,
    };

    match &import.kind {
        ImportKind::Simple => {
            self.bind_module(import.alias_or_name(), module)?;
        }
        ImportKind::Selective(names) => {
            for name in names {
                let symbol = module.export(name)?;
                self.bind_symbol(name, symbol)?;
            }
        }
        ImportKind::Glob => {
            for (name, symbol) in module.exports() {
                self.bind_symbol(name, symbol)?;
            }
        }
    }

    Ok(())
}
```

15.5 Type Checker

The type checker verifies type correctness and performs type inference.

15.5.1 Type Representation

```
enum Type {
    // Primitives
    Int, I8, I16, I32, I64,
    UInt, U8, U16, U32, U64,
```

```
    Float, F32, F64,
    Bool,
    String,

    // Composites
    Array(Box<Type>),
    Map(Box<Type>, Box<Type>),
    Tuple(Vec<Type>),
    Option(Box<Type>),

    // Named
    Struct(StructId),
    Enum(EnumId),

    // Functions
    Function(Vec<Type>, Box<Type>),

    // Generics
    TypeVar(TypeVarId),
    Generic(GenericId, Vec<Type>),

    // Special
    Never,
    Error,
}
```

15.5.2 Type Inference

```
fn infer_expr(&mut self, expr: &Expr) -> Result<Type,
    TypeError> {
    match expr {
        Expr::Literal(lit) => Ok(self.literal_type(lit)),

        Expr::Ident(name) => {
            self.lookup_type(name)
        }

        Expr::Binary(lhs, op, rhs) => {
            let lhs_ty = self.infer_expr(lhs)?;
            let rhs_ty = self.infer_expr(rhs)?;
            self.binary_result_type(op, &lhs_ty, &rhs_ty)
        }

        Expr::Call(func, args) => {
            let func_ty = self.infer_expr(func)?;
            match func_ty {
                Type::Function(params, ret) => {
                    self.check_args(&params, args)?;
                    Ok(*ret)
                }
                _ => Err(TypeError::NotCallable(func_ty)),
            }
        }

        // ... other cases
    }
```

```
}
```

15.5.3 Unification

```
fn unify(&mut self, a: &Type, b: &Type) -> Result<Type,
    TypeError> {
     match (a, b) {
         // Identical types
         (a, b) if a == b => Ok(a.clone()),

         // Type variable
         (Type::TypeVar(id), t) | (t, Type::TypeVar(id)) => {
             self.bind_type_var(*id, t.clone());
             Ok(t.clone())
         }

         // Option types
         (Type::Option(a), Type::Option(b)) => {
             Ok(Type::Option(Box::new(self.unify(a, b)?)))
         }

         // Arrays
         (Type::Array(a), Type::Array(b)) => {
             Ok(Type::Array(Box::new(self.unify(a, b)?)))
         }

         // Mismatch
         _ => Err(TypeError::Mismatch(a.clone(), b.clone())),
     }
}
```

15.6 AI Phase

The AI phase expands `ai` blocks into regular functions.

15.6.1 AI Block Processing

```
fn process_ai_block(&mut self, ai: &AiBlock) -> Result<FnDecl,
    AiError> {
     // 1. Generate cache key
     let key = self.cache_key(ai);

     // 2. Check cache
     if let Some(cached) = self.cache.get(&key) {
         return self.parse_cir(cached);
     }

     // 3. Prepare context
     let context = AiContext {
         signature: self.format_signature(ai),
         types: self.collect_types(ai),
         intent: ai.intent.clone(),
     };
```

```
16
17      // 4. Invoke backend
18      let cir = self.backend.generate(&context)?;
19
20      // 5. Validate CIR
21      self.validate_cir(&cir, ai)?;
22
23      // 6. Cache result
24      self.cache.set(&key, &cir);
25
26      // 7. Convert to AST
27      self.cir_to_ast(&cir)
28  }
```

15.6.2 Backend Interface

```
1   trait AiBackend {
2       fn generate(&self, context: &AiContext) -> Result<Cir,
            AiError>;
3       fn name(&self) -> &str;
4       fn is_available(&self) -> bool;
5   }
6
7   struct LlamaCppBackend {
8       model_path: PathBuf,
9       context_size: usize,
10      // ...
11  }
12
13  struct OllamaBackend {
14      host: String,
15      model: String,
16      // ...
17  }
18
19  struct AnthropicBackend {
20      api_key: String,
21      model: String,
22      // ...
23  }
```

15.7 Lowering

Lowering transforms the typed AST into HIR (High-level IR).

15.7.1 HIR Structure

```
1   struct HirModule {
2       functions: Vec<HirFunction>,
3       types: Vec<HirType>,
4       constants: Vec<HirConst>,
5   }
6
```

```
struct HirFunction {
    name: Symbol,
    params: Vec<HirParam>,
    return_type: HirType,
    body: Vec<HirStmt>,
    locals: Vec<HirLocal>,
}

enum HirStmt {
    Assign(LocalId, HirExpr),
    Store(HirPlace, HirExpr),
    Call(Option<LocalId>, FnId, Vec<HirExpr>),
    If(HirExpr, BlockId, Option<BlockId>),
    Loop(BlockId),
    Break(Option<LoopId>),
    Continue(Option<LoopId>),
    Return(Option<HirExpr>),
}
```

15.7.2 Desugaring

```
// Pipe operator
// a | f | g  -->  g(f(a))
fn desugar_pipe(&mut self, pipe: &Pipe) -> HirExpr {
    self.build_call(
        pipe.rhs,
        vec![self.desugar_expr(&pipe.lhs)]
    )
}

// String interpolation
// "Hello, ${name}!"  -->  "Hello, " + to_string(name) + "!"
fn desugar_interpolation(&mut self, s: &InterpolatedString) ->
   HirExpr {
    let parts: Vec<HirExpr> = s.parts.iter().map(|part| {
        match part {
            Part::Literal(s) => HirExpr::StringLit(s.clone()),
            Part::Expr(e) =>
               self.build_to_string(self.lower_expr(e)),
        }
    }).collect();

    self.build_string_concat(parts)
}

// For loop
// for x in items { ... }  -->  iterator pattern
fn desugar_for(&mut self, f: &ForLoop) -> Vec<HirStmt> {
    let iter = self.build_iter_call(&f.iterable);
    let loop_var = self.fresh_local();

    vec![
        HirStmt::Assign(loop_var, iter),
        HirStmt::Loop(self.build_block(vec![
            HirStmt::Assign(f.var,
               self.build_next_call(loop_var)),
```

```
            HirStmt::If(
                self.build_is_some(f.var),
                self.lower_block(&f.body),
                Some(self.build_break_block()),
            ),
        ])),
    ]
}
```

15.8 Code Generation

Code generation uses Cranelift to produce native code.

15.8.1 Cranelift Integration

```
fn codegen_function(&mut self, func: &HirFunction) -> Result<(), CodegenError> {
    let mut builder = FunctionBuilder::new(&mut self.ctx.func, &mut self.builder_ctx);

    // Create entry block
    let entry = builder.create_block();
    builder.append_block_params_for_function_params(entry);
    builder.switch_to_block(entry);

    // Generate parameters
    for (i, param) in func.params.iter().enumerate() {
        let val = builder.block_params(entry)[i];
        self.locals.insert(param.id, val);
    }

    // Generate body
    for stmt in &func.body {
        self.codegen_stmt(&mut builder, stmt)?;
    }

    builder.seal_all_blocks();
    builder.finalize();

    Ok(())
}

fn codegen_expr(&mut self, builder: &mut FunctionBuilder, expr: &HirExpr) -> Value {
    match expr {
        HirExpr::IntLit(n) => builder.ins().iconst(types::I64, *n),

        HirExpr::Binary(lhs, op, rhs) => {
            let l = self.codegen_expr(builder, lhs);
            let r = self.codegen_expr(builder, rhs);
            match op {
                BinOp::Add => builder.ins().iadd(l, r),
                BinOp::Sub => builder.ins().isub(l, r),
                BinOp::Mul => builder.ins().imul(l, r),
```

```
                BinOp::Div => builder.ins().sdiv(l, r),
                // ...
            }
        }

        HirExpr::Call(func, args) => {
            let args: Vec<Value> = args.iter()
                .map(|a| self.codegen_expr(builder, a))
                .collect();
            let call = builder.ins().call(func.ir_ref, &args);
            builder.inst_results(call)[0]
        }

        // ...
    }
}
```

15.9 Project Structure

```
haira/
  Cargo.toml
  src/
    main.rs               # CLI entry point
    lib.rs                # Library entry point

  crates/
    haira-lexer/          # Lexical analysis
    haira-parser/         # Parsing
    haira-ast/            # AST definitions
    haira-resolver/       # Name resolution
    haira-types/          # Type system
    haira-typeck/         # Type checking
    haira-ai/             # AI integration
    haira-cir/            # CIR handling
    haira-hir/            # High-level IR
    haira-codegen/        # Code generation
    haira-driver/         # Compilation driver
    haira-stdlib/         # Standard library
```

15.10 Build Process

```
# Full build
haira build main.haira

# Debug build
haira build main.haira --debug

# Release build
haira build main.haira --release

# Check without codegen
haira check main.haira
```

```
# Run directly
haira run main.haira

# REPL
haira repl
```

15.11 Error Messages

```
error[E0001]: type mismatch
  --> src/main.haira:15:12
   |
15 |     return "hello"
   |            ^^^^^^^ expected `int`, found `string`
   |
   = note: expected type `int`
              found type `string`

error[E0015]: undefined variable
  --> src/main.haira:20:5
   |
20 |     println(undefined_var)
   |             ^^^^^^^^^^^^ not found in this scope
   |
   = help: did you mean `undefined_value`?

error[E0042]: missing return statement
  --> src/main.haira:25:1
   |
25 | fn calculate(x: int) -> int {
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^ this function must return a
      value
   |
26 |     x * 2
   |     ----- help: consider adding a `return`: `return x * 2`
```

Appendix A

Reserved Keywords

The following identifiers are reserved as keywords and cannot be used as identifiers:

```
ai          and         as          break       catch
chan        continue    defer       else        enum
false       fn          for         from        if
import      in          match       nil         not
or          return      select      spawn       struct
true        try         type        while
```

Appendix B

Operator Precedence

Operators are listed from highest to lowest precedence:

Precedence	Operators	Associativity
1 (highest)	`() [] .`	Left
2	`!` `-` (unary) `not`	Right
3	`*` `/` `%`	Left
4	`+` `-`	Left
5	`..` `..=`	None
6	`|` (pipe)	Left
7	`<` `>` `<=` `>=`	Left
8	`==` `!=`	Left
9	`and` `&&`	Left
10	`or` `||`	Left
11 (lowest)	`=` `+=` `-=` `*=` `/=`	Right

Appendix C

Standard Library Reference

Quick reference for standard library modules. See Chapter 10 for full documentation.

Module	Description
`io`	Input/output operations
`string`	String manipulation
`math`	Mathematical functions
`array`	Array operations
`map`	Map/dictionary operations
`json`	JSON encoding/decoding
`http`	HTTP client/server
`fs`	File system operations
`os`	Operating system interface
`time`	Time and duration
`crypto`	Cryptographic functions
`regex`	Regular expressions
`net`	Network primitives

Appendix D

CIR Schema

Complete JSON schema for the Canonical Intermediate Representation (CIR). See Chapter 13 for semantics.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "CIR",
  "type": "object",
  "required": ["version", "body"],
  "properties": {
    "version": { "const": "1.0" },
    "body": { "$ref": "#/$defs/statement_list" }
  }
}
```

(Full schema in Chapter 13)